

**CONTEXT-ORIENTED ML MODEL
PREDICTION AND
ADAPTIVE SELECTION SYSTEM USING LLMS**

Semester:IV

MASTER OF TECHNOLOGY

By:

AARSEE TYAGI (2403030020)



Department of Computer Science and
Engineering Jaypee Institute of Information
Technology Noida, Sector 62, Uttar Pradesh

January 2026 – June 2026

COURSE NAME: DISSERTATION

TABLE OF CONTENT

PREFACE	i
ACKNOWLEDGEMENT	ii
DECLARATION	iii
SUPERVISOR'S CERTIFICATE	iv
ABSTRACT	v
CHAPTER 1 INTRODUCTION	1
1.1 Introduction to AutoML	1
1.2 Motivation	2
1.3 Problem Statement	5
1.4 Research Objectives	5
1.5 Scope of Work	9
1.6 Application of Proposed System	11
1.7 Contemporary Challenges	13
1.7.1 Computational Cost in AutoML	14
1.7.2 Lack of Context-Aware Recommendations	14
1.7.3 Scalability and Explainability Issues	15
1.8 R&D Problems in Focus Area	17
1.9 Contributions of Proposed Research	19
CHAPTER 2 LITERATURE REVIEW	22
2.1 Integrated Summary	22
2.2 Identified research gaps	29
2.3 Objectives of Proposed Work	31
CHAPTER 3 ANALYSIS, DESIGN AND MODELLING	32

3.1 Introduction	32
3.2 Proposed System Overview	33
3.3 System Architecture	34
3.4 Workflow of Proposed System	37
3.5 Dataset Upload and Metadata Extraction	40
3.6 Target Selection and Feature Exclusion	43
3.7 LLM-Based Recommendation Engine	45
3.8 Prompt Engineering Strategy	46
3.9 Automated Model Training Pipeline	47
3.10 Knowledge Base and Continuous Learning	48
3.11 Dashboard and Analytics Module	50
3.12 Data Flow Diagram (DFD)	51
3.13 Functional Requirements Risk Analysis	51
3.14 Non-Functional Requirements	52
3.15 Assumptions	53
3.16 Risk Analysis	53
CHAPTER 4 IMPLEMENTATION AND RESULTS	54
4.1 Introduction	54
4.2 Development Environment and Tools Used	55
4.3 Frontend Implementation using React	56
4.4 Backend Implementation using FastAPI	58
4.5 PostgreSQL Database Integration	60
4.6 Recommendation and Training Workflow	63
4.7 Experimental Setup	64
4.8 Evaluation Metrics	65
4.9 Experimental Results and Analysis	66
4.9.1 Model Recommendation Accuracy	67
4.9.2 Comparison with Traditional AutoML	68

4.9.3 Computational Cost Reduction	68
4.9.4 Adaptive Recommendation Performance	69
CHAPTER 5 CONCLUSION AND FUTURE SCOPE	70
5.1 Conclusion	70
5.2 Summary of Contributions	71
5.3 Limitations	73
5.4 Future Scope	74
5.4.1 GPT-Based Recommendation Enhancement	75
5.4.2 Continuous Learning and Fine-Tuning	75
5.4.3 Joint project with CRFS for the Multi-Domain Benchmark Dataset	76
5.4.4 Explainable AI-Based Recommendations	77
5.5 Overall Closing Remarks	77
APPENDICES	79
A- Dataset Details	79
B- GitHub Link	80
C- References	81

LIST OF ACRONYMS AND ABBREVIATION

AI	Artificial Intelligence
AutoML	Automated Machine Learning
API	Application Programming Interface
CSV	Comma Separated Values
CPU	Central Processing Unit
GPU	Graphics Processing Unit
CAAFE	Context-Aware Automated Feature Engineering
CASH	Combined Algorithm Selection and Hyperparameter Optimization
DFD	Data Flow Diagram
EDCA	Evolutionary Data-Centric AutoML
F1 Score	Harmonic Mean of Precision and Recall
FastAPI	Fast Application Programming Interface
GPT	Generative Pre-trained Transformer
HPO	Hyperparameter Optimization
LLM	Large Language Model
ML	Machine Learning
MLCopilot	Machine Learning Copilot
MSE	Mean Squared Error
NAS	Neural Architecture Search
NLP	Natural Language Processing
NoSQL	Not Only SQL
PostgreSQL	Object-Relational Database Management System
R&D	Research and Development
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
SQL	Structured Query Language
TPOT	Tree-based Pipeline Optimization Tool

LIST OF FIGURES

Figure Number	Caption	Page Number
1	Architectural Diagram	35
2	End to End Flow	38
3	DFD	51
4	Component Interaction	59
5	Script for Data Profiling(a)	61
6	Script for Data Profiling(b)	61
7	Script for Data Profiling(c)	62
8	Script for Data Profiling(d)	62

LIST OF TABLES

Table Number	Title	Page Number
1	Integrated Summary	27
2	Dataset Details	79

PREFACE

This dissertation has been prepared as part of the requirements for the M.Tech at [Jaypee Institute of Technology, Noida]. The primary objective of this work is to explore and analyse **“Context-Oriented ML model prediction and adaptive selection system using LLMS”**.

The report presents the motivation behind choosing this topic, surveys the existing literature, identifies research gaps, and proposes an approach to address the challenges observed in current systems. The study also includes the design, implementation, and evaluation of the proposed methodology.

The work has been carried out with the intention of contributing to the understanding and development of secure, reliable, and efficient solutions in this domain. Every effort has been made to ensure clarity, accuracy, and completeness in presenting the findings.

I hope this dissertation proves useful to students, researchers, and practitioners interested in this field.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **Dr. Diksha Chawla** for their valuable guidance, continuous support, and constructive feedback throughout the duration of this dissertation. Their expertise and encouragement greatly helped me in completing this work.

I am also thankful to the faculty members of the **Department of Computer Science and Engineering of Jaypee Institute of Technology, Noida** for providing the essential resources and a conducive learning environment for my research.

My heartfelt thanks to my family and friends for their constant motivation, patience, and emotional support, which kept me going throughout this journey.

Finally, I would like to extend my gratitude to everyone who directly or indirectly contributed to the successful completion of this dissertation.

DECLARATION BY THE SCHOLAR

I hereby declare that the work reported in the MTech Dissertation entitled “**Context-Oriented ML model prediction and adaptive selection system using LLMS**”. submitted at **Jaypee Institute of Information Technology, Noida, India** is an authentic record of my work carried out under the supervision of **Dr. Diksha Chawla** I have not submitted this work elsewhere for any other degree or diploma. I am fully responsible for the contents of my M.Tech dissertation.

Aarsee Tyagi

Department of Computer Science and Engineering

Jaypee Institute of Information Technology, Noida, India

23.05.26

SUPERVISOR'S CERTIFICATE

This is to certify that the work reported in the M.Tech Dissertation entitled “**Context-oriented ML model prediction and adaptive selection system using LLMS**” submitted by **Aarsee Tyagi** at **Jaypee Institute of Information Technology, Noida, India** is a bonafide record of her original work carried out under my supervision. This work has not been submitted elsewhere for any other degree or diploma.

Dr. Diksha Chawla

Assistance Professor (Senior Grade)

Jaypee Institute of Technology Noida, India

23.05.26

Prof. Sikha Mehta (Head of Department)

Department of Computer Science & Engineering and Information Technology

Jaypee Institute of Information Technology, Noida, India.

23.05.26

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO AUTOML

Machine Learning (ML) is a vital aspect of modern computing and Artificial Intelligence (AI). It enables computer programs to identify patterns within a set of data and subsequently makes inferences and/or decisions about the data, without the need to explicitly program the procedure for each individual case. In ways, in the past decade, the fact that a machine can now process, manipulate, and compute data has modified the industrial process of data from the physical realm to the virtual realm in order to tackle complex tasks. Other examples of machine learning in different fields include, in medicine, predictive and prognostic diagnosis of diseases; in finance, detection of fraudulent activities; in e-commerce, development of recommender systems; in cyber security, analysis of threats; and, in the field of transportation, smart traffic systems and autonomous vehicles.

Usually, Data Scientists would implement different algorithms and settings and then choose the best ones. Selecting test algorithms and different settings would be the norm for a Data Scientist. This consumed a lot of time and computing resources, and the results were very dependent on the skill and competency of the developer(s). In actual working environments, especially in small businesses, classrooms, and teaching environments, the users of the system had little knowledge of machine learning. This made it hard for them to use machine learning because it required the skills of a computer scientist.

Automated Machine Learning (AutoML) systems were developed to tackle some of the problems associated with the traditional machine learning workflows. AutoML applications depend less on human intervention. At its heart, the aim of AutoML is to democratize the ML development process by the automation of several tasks such as the selection of the most appropriate model, hyperparameter tuning, feature engineering, and the design of the pipelines.

AutoML frameworks today are capable of automatically generating the ML pipelines of the highest performance. Examples of such frameworks include AutoSklearn, TPOT, H2O AutoML, and AutoGluon. The search for the best combinations of models and hyperparameters

are aided by several techniques including Bayesian Optimization, Grid Search, Random Search, Meta-Learning, and Evolutionary Algorithms.

When it comes to AutoML, of course, there are advantages and disadvantages. Even with the degree of automation of ML brought about by the various AutoML offerings, several of the frameworks are still deeply reliant on experimentation. The selection of the best performing model is achieved through testing of a wide range of candidate model designs. This is often very complicated and time-consuming, particularly with large and dense datasets.

The advancement of Large Language Models (LLMs) provides additional opportunities for intelligent automations of the entire machine learning (ML) workflows. An example of this is the GPT and Gemini models, which have the ability to read, understand, and process text. This opens a new synergy between ML automation and the ability to reason.

The topic for the dissertation is the development of an ML Model Prediction and Adaptive Selection System for a Context-oriented System that utilizes LLM. This system uses AutoML in a much less taxing manner, attempting to suggest the most appropriate machine learning model based on the data set's attributes and the metadata and context. This approach attempts to save computational resources, enhance the quality of recommendations, and achieve a model selection system that is easier to understand.

This integration of machine learning automation with LLM reasoning marks an important milestone in the evolution of artificial intelligence systems, which are becoming increasingly adaptive, intelligent, and efficient. This thesis proposes a solution which moves toward this paradigm shift by offering a system combining contextual awareness, automatic evaluation, and learning functionalities to optimize machine learning processes.

1.2 MOTIVATION

With the current technological development and the volume of data available, the use of machine learning algorithms has greatly increased across numerous fields. Currently, enterprises apply machine learning techniques to develop practical solutions to problems related to, but not limited to, predicting diseases, fraud detection, developing recommendation systems, analyzing customers, cybersecurity, and automation. Although there are numerous ways to apply machine learning nowadays through cloud services and open source libraries, the construction of machine learning systems is quite complicated and time consuming.

The hardest part of the machine learning process is choosing the best algorithm for the data set. Algorithms behave differently with respect to different kinds of data sets. Properties of the data like its volume, kinds of features, missing values, class imbalance, high dimensionality, and noise can greatly influence the quality of the solution provided by the model. The choice of an algorithm and hyperparameters for developers requires trials and errors.

Experimentation in real-world scenarios is generally done through some form of trial and error. The Data Scientists create a model, check its accuracy using the various measures, tweak the parameters and repeat the process multiple times. This process consumes a great deal of time and computations and might not be possible for someone with experience. As the complexity of the task rises, the difficulty in solving it for new users and non-experts becomes greater since they might not be aware of all the algorithms necessary for certain types of data sets.

These methods improve predictive efficiency, yet they try out many different combinations of models until they get to the best one. These sorts of experiments will become more and more costly as the size and complexity of the data sets increase. At times the system tries to apply algorithms that cannot work for the data set. This results in wasted computation time.

Another motivation for this research was related to the rising interest in the concept of sustainable computing and the creation of energy-efficient intelligent technologies. Re-training models of machine learning on large datasets requires a lot of resources and is energy-consuming. Conducting less experimentation may lead to greater efficiency and more sustainable computing. Therefore, it becomes especially important to develop intelligent models capable of facilitating the decision-making process in model selection in an effective way.

This became a really interesting problem to work with. Instead of blindly analyzing all the possible options of machine learning algorithms, one might think about the characteristics of a particular dataset and come up with intelligent guesses about the most applicable algorithms. Having this information at hand before the model is trained allows us to narrow the field of operations and eliminate unnecessary computations.

As an example, some types of datasets can be better suited for tree-based models, while other datasets can be better fit for ensembling methods and neural networks. In many cases, a choice could be made based on an expert decision, which would rely primarily on experience and

intuition. One of the main goals of this project would be to create a solution that would emulate the intelligent decision-making process by combining LLM and automation.

Another important reason is that many existing AutoML solutions are not transparent. Most of these systems would provide users with the recommended model choice but without explaining the reasons for that decision. In most cases, users will not know what logic was used when choosing the algorithm. This lack of transparency is especially problematic in industries such as health care and finance, where explainability is very important.

Moreover, another motivation for carrying out this research was the increasing recognition of the significance of sustainable computing and energy efficiency in AI systems. Training machine learning models on large datasets consumes much energy and compute power. This way is not only efficient but also more sustainable when it comes to using computers in the process. Thus, it is necessary to consider a certain intelligent approach rather than taking a brute-force solution to the problem.

This was a great opportunity for conducting research. In order to examine all machine learning models is not always required – it is possible to evaluate the data set properties and the domain it is used for, and thus forecast which machine learning model would suit better. Knowing beforehand the characteristics of the data set that one is trying to find will help narrow down the scope of the investigation.

However, some data sets will fit the tree model better, while other data sets will suit ensemble methods and/or neural networks better. Usually, this depends on the intuition or experience of the data scientist. This project attempts to create an application that will attempt to perform intelligent recommendation similar to that performed by data scientists with the assistance of LLM and automation of ML workflows.

The first motivation for choosing the topic of intelligent automated machine learning workflow is related to the fact that many existing auto-machine learning platforms lack transparency and interpretability. Most auto-machine learning algorithms provide recommendations to the user. However, they do not provide explanations on how these recommendations were obtained. Most users will get the results of a certain process as predictions, having no idea about the reasoning behind the process. The proposed system attempts to provide explanations to users by integrating LLM-based reasoning within the process of generating recommendations.

The concept of adaptive learning systems also inspires the proposed research. All recommendation systems currently available do not have any possibility of further improvement and optimization after implementation and are not dynamic. In other words, there will be a knowledge base where the results of the recommended decisions, as well as the results of training according to this advice and interaction with the user, will be stored. Thus, the system will always be able to adjust its recommendations in the future based on knowledge received.

It is not only the pursuit of improving the accuracy of predictions that is behind this dissertation proposal. The final aim is to create an effective, intelligent, adaptive and optimized recommendation system, which will use the opportunities of large language models and automatic machine learning. The research itself will be expected to advance the creation of intelligent software engineering systems with artificial intelligence, improve performance, make models explainable and selectable.

1.3 PROBLEM STATEMENT

End-to-end machine learning modelling is often resource heavy and does not always work best for the particular dataset, which is typical in modern software engineering and sustainable computing. We introduce a novel framework, supported by an LLM, for dynamically understanding the nature of a dataset and user requirements to efficiently recommend optimal model types. Our system will evolve over time and tackle minimizing trial and error to reduce the waste of computation because of continuous feedback loops, empirical results and adaptive knowledge bases. This is consistent with sustainable computing through the streamlined selection of models, but is also a contribution to the computer science automation research area. We fill the gap of intelligent and context-aware model recommendations by providing an ongoing and data-driven solution for practitioners.

1.4 RESEARCH OBJECTIVES

The proposed research has the following major objectives.

1.4.1. To Analyze Dataset Characteristics and Metadata

The aim of this research is to study the structural and statistical properties of uploaded databases. The performance of machine learned models depends heavily on the features of the

dataset, like the number of data points, the distribution of values in features, feature types, missing values, imbalance ratio, dimensionality and noise.

The designed system is intended to automatically extract and process the metadata of the dataset to understand the nature of the data before the model training process is started. The system could analyze the characteristics of the datasets beforehand and make more informed and context-aware recommendations rather than relying on the brute force experiment.

Another goal for this objective is to decrease the amount of unnecessary computing by pruning the search space according to the properties of the data set.

1.4.2. To Create Context-Aware Machine Learning Recommendation System

One of the major goals of this research is to be able to construct an intelligent recommendation system that can predict appropriate machine learning algorithms based on the context and not by trying them all out.

The proposed framework can leverage Large Language Models to interpret dataset information, metadata summaries, sample records and optional user instructions. The aim is to simulate experienced data scientists who tend to choose algorithms depending on their past experience and knowledge about the data sets.

The system attempts to evaluate the performances of all possible models in a blind manner, but rather to intelligently prioritize those algorithms that are more likely to be successful for a given data set. This enhances the efficacy of recommendations, and also lessens the computational overhead.

1.4.3. Develop effective integration strategies for Large Language Models and Machine Learning Workflows.

One of the main goals of this dissertation is to examine the feasibility of using a Large Language Model for integrating into machine learning automation systems.

While LLMs are effective in tasks like reasoning, code generation, and understanding context, they are not explored as much in the field of intelligent machine learning model recommendation. The goal of this proposed research is to fill this gap by embedding reasoning using LLM into the machine learning pipeline.

The system will employ LLMs to:

- interpret dataset characteristics,
- generate contextual recommendations,
- explain the reasons for choosing models, and
- and enable workflows for adaptive recommendations.

This goal helps achieve the development of more intelligent and human-caring AI supported systems.

1.4.4. To Minimize Number of Calculation and Experiment

Exhaustive search-based optimisation methods are one of the key challenges of traditional AutoML systems, due to their high computational costs.

When applying existing frameworks, many machine learning models and hyperparameter sets are trained and tested before good solutions are found; and the number of models and sets is usually very large. This process will make execution time, memory usage and energy consumption higher.

The proposed research will seek to alleviate this computational burden by enhancing recommendation systems to be context oriented. The advantage of such a process is that it reduces the number of experiments that are made by the system and makes the overall process more efficient.

This goal is also in line with the sustainable and resource-efficient computing concept.

1.4.5. To Develop Adaptive Recommendation and Learning Mechanism

The other aim of this dissertation is the development of a framework for recommendations that will be continuously improved by learning.

Currently, there are very few recommendation systems that significantly tune their recommendation after they are deployed. The proposed system proposes a mechanism to store the previous recommendations, training results, evaluation metrics and outcomes to a knowledge base.

This information can be used by the system to refine the future recommendations and cater to changes in the data sets and users' needs. The adaptive learning capability is designed to enhance the usability and intelligence of the framework for future applications.

1.4.6. To make Model Recommendation Explanatory and Transparent. Make Model Recommendation Explanatory and Transparent

Existing machine learning automation systems work like black boxes and the user only gets the outputs without knowing how they are obtained.

The goal of the proposed research is to create explanations for machine learning recommendations in a way that is human-readable and interpretable. The framework can provide explanations for the suitability of specific algorithms for a given dataset, because LLMs have the ability to generate contextual reasoning in natural language.

This goal is especially highly valued by users with limited machine learning skills, and in areas where interpretability is important.

1.4.7. To automate the entire model recommendation workflow from end to end

Another goal of this research is to streamline and automate the machine learning process by implementing an automatic system.

The framework being proposed to integrate the following:

- dataset upload,
- metadata extraction,
- target column selection,
- feature exclusion,
- recommendation generation,
- automated model training,
- evaluation,
- artifact generation,
- and result visualization
- All within one smart platform.

This cohesive development process can minimize development complexity and make it more user-friendly for both technical and non-technical users.

1.4.8 To analyze the performance of the system proposed.

The other key goal of this research is to practically assess the performance and effectiveness of the suggested recommendation framework with benchmark datasets.

The system will be tested with various datasets from various domains to analyse:

- recommendation quality,
- prediction performance,
- computational efficiency,
- scalability,
- and adaptive learning behaviours.

The proposed framework is compared with the traditional AutoML approaches by using the evaluation metrics like accuracy, precision, recall, F1 score, execution time, and resource utilization.

1.4.9. To contribute to an intelligent and sustainable AI system

This dissertation aims at advancing the efforts for intelligent, adaptive, explainable and resource-efficient AI-assisted software engineering systems in general. The proposed research aims to combine contextual reasoning and automation capabilities provided by machine learning, to transcend from search-based approaches to the more human-centered intelligent systems, which can support the user better.

The framework does not focus solely on conversational applications, but also on practical decision-support applications across the sphere of machine learning and software engineering, to help showcase the potential to apply LLM to tasks beyond conversation.

1.5 SCOPE OF WORK

In this research, focus is on designing and developing a Context Oriented ML Model Prediction and Adaptive Selection System (nnCOMPASS) with the help of Large Language Models (LLMs). The goal of the proposed system is to introduce the possibility of simplifying the

machine learning model selection process, combining the concepts of the analysis of selected datasets, contextual reasoning, automatic training and adaptive recommendation mechanisms in a single framework.

The research mainly focuses on developing an intelligent recommendation system that can analyse uploaded datasets and recommend suitable machine learning algorithm based upon the characteristics of the dataset. Important metadata, like size of the dataset, the type of features its data contains, the presence of missing values, statistical summaries and structural information about a dataset is retrieved from the system and used to understand the structure of the data before the training process actually starts.

One of the top challenges is embedding Large Language Models into the recommendation process. To recommend machine learning models at the metadata level, the LLM processes the metadata of the dataset and the different samples or optionally user instructions. The proposed system tries to smartly reduce the search space using the context awareness and reasoning as opposed to traditional AutoML systems which use exhaustive search techniques completely.

As part of the research, an automatic machine learning workflow is implemented to automatically train and auto-evaluate the recommended models using machine learning libraries. Tasks like selecting the target columns, exclusion of features, analysis of metadata and evaluation of models, and generation of the results are all made possible by the integrated system. Both frontend and backend implementation is distinctively covered. The dedicated frontend enables the transfer of datasets, the setting of training options, recommendations and the evaluations results. All the processing of the datasets, metadata extraction, integration of LLM, automatic model training, and database management is handled within the backend.

The research also evaluates the proposed framework with reference to the benchmark datasets from various domains to assess the effectiveness of the proposed framework. In addition to measuring the quality of the recommendations, the accuracy of the predictions, and the computational efficiency of the recommendation system, recommendation systems are also compared with traditional AutoML approaches to assess their performance.

In this work however, the focus remains upon networks of structured tabular data and on widely used machine learning algorithms. These are tasks beyond the scope of the dissertation, including the use of advanced deep learning architectures, processing unstructured data,

training models in real time in distributed environments, and deployment of these models in large deployments on cloud platforms. Likewise, rather than deployment of the whole system as a finished production, it takes the focus on recommending and selecting a model.

In summary, the proposed research primarily aims to develop a practical LLM+ML Automation-based intelligent framework for recommendations, making it more accurate, time-saving, and streamlined for the machine learning process.

1.6 APPLICATIONS OF PROPOSED SYSTEM

The proposed Context-oriented ML model prediction system with Adaptive Selection System using LLM can be used in many different applications such as machine learning in prediction, automation, and data driven decision processes. Given that intelligence is also the key aspect of the system with regard to model selection and automating workflows, it can prove useful even for non-technical users when building effective machine-learning applications, without too much manual work.

Among the significant applications in the proposed system, healthcare analytics are one of them. Medical centers collect vast quantities of patient data, including medical records, diagnostic reports, lab results, treatment histories, and more. There is a need for careful consideration and experimentation when deciding which machine learning algorithms are appropriate for prediction tasks, patient risk assessment or diagnosis assistance. The healthcare system can benefit the researchers and developers by analysing medical data sets and suggesting appropriate machine learning algorithms for predictive tasks, thereby saving unnecessary computational tasks.

One field of application which is of great interest is financial fraud detection and risk analysis. Machine learning systems are highly important for financial institutions in the battle against fraudulent transactions, anomaly detection, credit risk assessment, and user behavior analysis. Financial datasets are typically large in terms of features and classes, as well as large number of transactions. By suggesting appropriate models based on the characteristics of the dataset and streamlining fraud detection processes, the proposed framework can help analysts handle the growing complexity of fraud detection tasks. The proposed framework can aid the analysts by recommending the suitable models based on the characteristics that is present in the database and can upgrade the efficiency of fraud detection process flow.

The system could also be applied in the e-commerce and recommendation systems. As users surf the Web, they generate data about their interactions with the site, including browsing patterns, purchase history, ratings or any type of preference. The proposed system can be used in the field of education and academic study, by supporting students, researcher, and beginner developers in machine learning area, who might not have enough skills in machine learning. It is difficult for many users to determine which algorithms to employ for given sets of data and often requires trial and error experimentation to be repeated. The proposed system streamlines this process by making it more easily accessible and intelligently user-friendly, through intelligent recommendations and automated evaluation workflows, turning machine learning more accessible and user-friendly.

One more possible application field is Cybersecurity and Network monitoring. Whether it is intrusion detection, malware classification, anomaly detection, or security analysis, machine learning is now playing a pivotal role in that domain. Security data can be complex and constantly landscape-changing. Security data's commonly contain intricate patterns and constantly shifting dangers. The proposed system can help to choose a proper machine learning model for prediction problems in the field of securing by taking into consideration the structure of the data sets and the characteristics of its features.

The framework is applicable to smart manufacturing and automation in industry as well. Industrial sensors and machines collect data and logs that can be used for process optimization, quality analysis, and predictive maintenance using machine learning techniques. Proposed system can be implemented in industry, where it can automatically pick the suitable model and minimize the time spent on creating the predictive analytics solution.

In addition to specific domain application, the overall system can be used as an intelligent decision making-assisting tool for software development and data science application. The framework can be automated by organizations and developers within the same platform to enable quick analysis, model recommendations, model training and evaluation for a given dataset. This has a positive impact on the productivity and decreases manual experimentation.

In general, the proposed system is applicable to a wide range of areas where the choice of machine learning models and the automation of their workflow is crucial. The proposed framework's contextual reasoning machine-learning automation process will be more efficient, less computational intensive, and easier to develop intelligent data-driven applications.

1.7 CONTEMPORARY CHALLENGES

While machine learning and Automatic Machine Learning (AutoML) technologies have made great strides in the last few years, there remain a number of practical and technical hurdles in the machine learning model selection and machine learning workflow automation process. With the growth in the magnitude of the datasets and the complexity of the machine learning applications, traditional methods can become inefficient, large-scale, lack of interpretation, and lack of context awareness. Moreover, most recommendation systems focus on optimization, based in numbers and statistical performance. Their dataset selection of machine learning models is typically a purely technical process, without taking into account contextual, semantic, or human thinking aspects of the data sets. This reduces the effectiveness of the recommending process and the understanding of the recommendations.

Discussed below is some popular work in the current challenges with the existing machine learning and AutoML systems.

1.7.1 Computational Cost in AutoML

High computation cost of model optimization and selection is one of the major challenges in the current AutoML systems. Existing traditional AutoML frameworks are based on an extensive search strategy like Grid Search, Random Search, Bayesian Optimization and Evolutionary Algorithms. Those techniques do a variety of shuffles through different sets of algorithms and/or hyperparameters in order to determine which is the perfect one.

These methods can yield an accurate model; however, they will need a considerable amount of computing power and enough time for them to run. The search space becomes huge as the number of algorithms (and parameter set combinations, too) grows. Several machine learning models could need to be trained and tested upon several times in the system before the best one is found.

This process might be feasible for small amounts of data. The number of calculations becomes huge, though, with a large number of real-world records, say, thousands or millions, and it becomes a heavy burden. Often running multiple models in repeated training can use a lot of CPU and GPU resources, cause a lot of memory to be used and can significantly lengthen training time.

Another problem encountered with the exhaustive experiment is the wasting of resources. Many AutoML systems are still assessing models which may not be an appropriate model structure for the dataset because of the type of dataset. Some algorithms might be less effective in certain extreme cases, such as a data set of high- dimensional categorical features or highly imbalanced. Even so, they might be still be included in the search and judged in traditional ways, incurring computational overhead.

Especially in academic institutions, startups, and small organizations with restricted hardware resources, where powerful computing resources might not be accessible, this problem is becoming more crucial. Numerous users may struggle with computationally intensive workflows with AutoML and their execution time is too long.

As more and more attention has been given to sustainable computing, the need to decrease the computational workloads has become more prominent. Training and optimization of modern AI systems require a lot of energy. Attempting to repeat already-tried experiments that span wide ranges of search space leads to higher energy consumption, and inefficient usage of resources.

1.7.2 Lack of Context-Aware Recommendations

One of the other crucial challenges faced in the current machine learning based recommender system is the inference of absence of contextual understanding while selecting models. Traditional autoML frameworks mostly aim at statistical optimisation and powerful numerical evaluation scores but fail to take into account the context of the data.

The number of the features, missing values, unbalanced data, dimensionality, relationships among the features, and the domain knowledge of the data are common factors that experienced data scientists consider when making decisions about model selection. Human experts frequently use previous information about the problem and make their own judgments and guesses about what algorithm(s) to train, before the training process starts.

But the existing typical recommendation systems lack such context awareness. Rather, they consider model selection in most cases as an optimization exercise: they run several algorithms in sequence to eventually come up with a good performing model configuration. Therefore, the

models which are not likely to be able to produce good results for the set of data under consideration are often subject to systems evaluation.

Some algorithms is suitable for specific data structure like tabular, some for sparse data, some for high dimensional data, etc. Likewise, if the input dataset of images is highly imbalanced, certain algorithms and/or preprocessing can be used. In traditional AutoML systems, the contextual information is not always used correctly and intelligently to start the search process.

Another disadvantage is that a large majority of systems are mainly based on numerical metadata. Typically, they do not make use of semantic knowledge, data descriptions or instructions from the user or context information within the dataset while providing recommendations. This weakens the transformation power of recommendation frameworks to become an intelligent Assistant.

Large Language Models have brought in new opportunities for Contextual Reasoning for ML systems. LLMs can read natural-language descriptions, augment themselves using the relationships between different inputs, and make intelligent recommendations based on the context. But it is not yet as these features have been implemented into any tangible AutoML systems.

Machine learning workflows, without context-aware reasoning, can be inefficient and the experiments need to be repeated. It takes many hours of experimentation by hand for the user to arrive at an algorithm after the current characteristics of the data suggest other have been tried.

To resolve this issue, the proposed study comes up with a context-oriented recommendation framework: metadata from the datasets, sample records and user instructions (which would be optional) will be analysed with a LLM. To build a system that can provide contextually relevant and statistically meaningful, yet efficient recommendations.

1.7.3 Scalability And Explainability Issues

One more challenge that autoML and machine learning recommendation systems have to address is their ability of being scalable. Many of the traditional systems fail to perform optimally as data sets get larger in size and complexity. The dimensionality of the features,

missing values, and complexity of the relationships within large datasets make processing and training more challenging.

The scaling of most AutoML frameworks is poor because they require the repeated training of a number of models and evaluation. As the number of pages in a search space gets very large, it takes much longer to execute. This can slow down and find machine learning workflows impractical to use in the world where fast decisions matter.

Other scalability concerns come into play when systems are required to accomplish more than one usage, or multiple data sets, at the same time. As system loads rise, more and more it becomes challenging to manage a backend, train the models, manage the storage, and allocate resources. Optimizing the performance and stability of an automated solution in such cases is still a challenge for many automated frameworks.

Apart from scalability, explainability is increasingly becoming a significant problem of the ML systems. There are numerous existing AutoML systems which are black-box and deliver recommendations or prediction outcomes without revealing why they made a specific prediction or suggestion.

This transparency is hard to come by and presents a couple of issues. Recommendations won't be as well received when users cannot grasp the reasons behind the choice of a specific model. For industries like healthcare, financial services, cybersecurity and education, explainability is particularly critical as the actions, verdicts, and recommendations made by a machine learning system can have a direct impact on individuals and organizations.

Consider, for instance, a health care system that is automatically recommending a machine learning algorithm for prediction of the medical diagnosis of a patient to health care professionals; if this occurs, they might want to understand the reasoning behind the process of the recommending before implementing the algorithm in the future. Financial institutions also typically need interpretable systems for regulatory and auditing reasons, as does the industry of insurance. The insurance industry uses systems that are also interpretable for the same reason as financial institutions.

Unlike other traditional AutoML frameworks, most do not offer explanations in human-readable formats in the form of explanations of why certain algorithms might be more

appropriate with a particular dataset. This makes it difficult for people to understand what the system does and diminishes their belief in the automated system.

The LLMs could be one of the answers here as these models can produce natural language descriptions when queried with a context. Combining reasoning with LLM in recommendation systems enables the generation of more interpretable and transparent recommendations that people can easily understand.

The proposed framework has the aim of increasing the predictability, i.e., it generates human-readable reasoning for recommendations produced by the model. The framework is not just a black box optimization system, but strives to give understandable explanations which would enhance the ability of the users in understanding the decision of recommendations.

Much overall, significant problems with existing AutoML systems remain, including computational complexity, lack of understanding of the context, limitations in scalability, and lack of explainability. These challenges need to be tackled to enable smarter and smarter adaptable, efficient and user-friendly machine learning recommendation systems.

1.8 R&D PROBLEMS IN FOCUS AREA

Machine Learning, Automated Machine Learning (AutoML), and Large Language Models (LLM) research and development into all three of these areas have burgeoned in the past few years. While significant progress has been made, there are a number of key challenges that still need to be overcome for practical application of machine learning in workflows. Even though there has been some degree of automation of the existing systems, there are still several problems in the aspects of computational efficiency, intelligent decisions, adaptability, and explanations.

The lack of effective context-aware model recommendation systems is one of the focus area's main research areas. Typical AutoML frameworks focus on algorithmic methods that involve extensive search for hyperparameters that are used to train and test numerous ML models. Good predictive performance can be obtained with these approaches, albeit they require intensive computing resources and are more time consuming.

The second important issue is the lack of intelligent understanding of the context of the datasets before starting the model training process. One appealing way that experienced data scientists typically go about making decisions on what model to use—is by looking at certain characteristics of the data set, including feature types, missing values, class imbalance, and dimensionality. However, most of the currently available AutoML systems rely solely on numerical optimisation and statistical analyses. This causes systems to expend resources on their tests of inefficient algorithms on data sets.

The application of LLM with real-world applications on a machine learning recommendation systems is still an emerging research. Recent work with LLMs primarily includes code generation, prompt design, automatic documentation, or chatbots. Few research works have focused on intelligently recommending machine learning models and adaptively selecting them with the help of LLMs. This poses a significant research need, and not only the opportunity to build a smarter recommendation framework, but also more intelligent ones.

A second major issue identified in the focus area is the absence of adaptive learning mechanisms of the existing systems. Most recommendation systems are statically, and once deployed providing little change after deployment. Few effective cases studies exist on how to use historical training outcome and recommendation results, as well as user interactions to improve the individual effectiveness of future recommendations. As a result, systems do not develop in terms of learning from experience and out-of-the-lab use.

Another big challenge of modern-day AutoML systems concerns scalability. With growing size and complexity of the datasets, traditional search-based approaches slow down and consume more resources. In large-scale experiments, a strong hardware infrastructure might be necessary, which can result in steep demands on storage and hardware resources of individual users.

Explainability and transparency are a further important challenge. There are many automated recommender systems of which the user does not know how the recommendations were produced, or the rationale, and is just expected to accept the recommendation. This is not interpretable and makes users distrust them and makes them impractical in some areas such as the healthcare fields, finance, security etc.

Beyond the technical constraints, there is also a significant focus on sustainable computing and on-the-fly Artificial Intelligence systems that are resource-efficient. Unnecessary model training uses a lot of computational power and energy, as do repeated experiments. The focus of most existing systems is typically directly to disseminate content without paying attention to reducing computational "waste" via intelligent recommendation mechanisms.

An added challenge that comes into mind is the fragmented workflow in machine learning development. Each of the users has to use several tools one by one for dataset preprocessing, metadata analysis, model training, model evaluation, visualization and management of the artefacts during this workflow. This makes the development more complex, and ultimately productivity becomes lower.

These research and development challenges are being tackled by the proposed dissertation that introduces a Context-Oriented ML Model Prediction and Adaptive Selection System with LLM, which is a novel technique. The framework is a single intelligent system that integrates the above, therefore offering contextual reasoning, metadata analysis, automated training pipelines, adaptive learning components, as well as explainable recommendations.

The research will support advancing AI empowered software engineering systems with intelligent automation to improve machine learning processes, optimize resources, provide explainability, and be adaptable and scalable.

1.9 CONTRIBUTIONS OF PROPOSED RESEARCH

This is a proposal for research in the expanding realm of intelligent machine learning automation, suggesting a novel method centred around using context when recommending a Machine Learning pipeline using the use of Large Language Models with automated machine learning workflows. This dissertation contributes the most with creating a realistic system that can analyse the features in datasets and intelligently recommends the most appropriate machine learning models with minimized amount of computational work.

The integration of contextual reasoning in the selection of the set of machine learning models is one of the main impact of proposed work. In contrast to the typical AutoML systems based on exhaustive search approaches, the proposed framework employs LLM models to comprehend the information contained in the datasets meta data, sample records, and user

instructions, to develop suggestions. This brings in a smarter and more human-like method of modelling selection.

The other contribution of great importance is the adaptive recommendation mechanism. Recommendation results, indicators for measurement/evaluation and past training results are saved in a knowledge base. This is stored within the framework, which could be used for future recommendations and fine tuning purposes within the decision-making process. The adaptive capacity makes this system more feasible and flexible for a large-scale application in the long term.

The proposed research will also help to lower down the additional computation load on machine learning processes. The task is to reduce the size of the search space before the model is trained to lessen the amount of trial and error, and to make efficient use of the resources. This helps towards more sustainable and efficient computer-based AI systems.

As well another significant contribution is in the enhancement of explainability of Auto Recommendation Systems. There are numerous existing sources of models that give suggestions, but without mirroring to the models. Proposed work utilizes the LLLM's natural language model processing to create explanations, in natural language, that describe the reasons an algorithm might be suitable for a dataset. This enhances visibility and boosts users' confidence in the recommenders.

The research is also significant and enhances the previously described aspects by adopting several aspects of machine learning into a single platform. The proposed system integrates the following functions in an intelligent system: dataset upload, metadata extraction, selection of the target, feature exclusion, generation of recommendations and recommendations, automatic model training, model evaluation, generation of artifacts and visualization on the dashboard. This integration facilitates machine learning development, and can help both technical and non-technical users become more accustomed with the machine learning.

A key aspect of this work is Large Language Models (LLMs) in a relatively unexplored territory of machine learning automation. Most of the existing studies either in the field of code generation or prompt optimization with LLM, the suggested research works on the use of LLM for intelligent machine learning model prediction and adaptive recommendation system.

In addition, the dissertation is relevant to the field of software engineering research, as the proposed AI-assisted systems hold potential for enhancing the automation, decision-making, and development processes in software engineering. The framework emphasizes the real life usage of the intelligent reasoning systems in conjunction with machine learning automation.

In general, the proposed research projects an intelligent reduction in the complexity of IOT development and intelligent enhancement of the workflow of automation with the development of adaptive machine learning systems that are explainable, efficient and context-aware.

CHAPTER 2

LITERATURE REVIEW

2.1 Integrated Summary

There have been recent breakthroughs in automated machine learning (AutoML), meta-learning, and large language models (LLMs), all of which have made a notable impact on intelligent systems in machine learning model selection and pipeline automation. Building an automatic pipeline for various steps in the machine learning process, such as algorithm selection, feature engineering, pipeline optimization and even model debugging have been tried by several studies. This section discusses some of the important work in these fields.

1. Algorithm Selection on a Meta Level (2021)

Tornede et al. (2021) study the question of which algorithm is the most appropriate to solve some instance of a problem. The authors suggest a meta-learning framework which is higher level and fuses several algorithm selection approaches. The traditional algorithm selection techniques are almost based on predicting algorithm performance on the basis of the characteristic of the dataset. The proposed meta-level framework, however, comes with an ensemble of algorithm selectors, which collectively select the best algorithm for an individual instance.

The authors show that using multiple algorithm selectors greatly enhances performance over a single selector. Their experiments indicate that ensemble methods like boosting and bagging can be used to effectively leverage complementary strengths of different selection methods. The study also points out the need to make proper selection of the algorithm at the instance level, and shows that no single algorithm performs optimally for all problem instances, according to the No Free Lunch theorem.

The proposed approach boosts the algorithm selection performance, but heavily depends on the historical performance data and needs huge training datasets that can be used to construct accurate meta-models.

2. MANAS: Mining Software Repositories to Assist AutoML (2022)

Nguyen et al. in their paper “MANAS: Mining Software Repositories to Assist AutoML” (2022) studied how to use software repository mining to improve neural architecture search

(NAS). The authors note that many developers develop machine learning models for comparable tasks and save them in a repository, like GitHub. The proposed MANAS system does not re-invent the neural architecture search from scratch, but instead, it extracts the model architectures from the model repository as starting points for the neural architecture search process.

The framework examines the characteristics of common neural network architectures, and the typical modifications applicable by the developers to enhance the performance of the model. These are then employed as mutation operators for NAS. The experimental results reveal that the models generated based on the MANAS framework have much less parameters and faster training speed compared with the models generated based on traditional NAS techniques.

This strategy yields a more efficient search, but is directed towards deep learning architectures and might not apply to other machine learning algorithms.

3. Which is the Best Model for My Data? (2022)

In the paper “Which is the Best Model for My Data?” In a recent paper (2022), Nápoles et al. introduce a meta-learning strategy for the prediction of the most appropriate machine learning model for a particular dataset. The authors devise a system that extracts statistical meta-features from the datasets and train a meta-classifier that can predict which algorithm would be most likely to perform well.

It presents a collection of 62 meta-features that describe the distribution, correlation and complexity of the datasets. Employing these features, a meta-dataset is created that associates properties of the datasets with the performance of the algorithms. The experimental results validate the proposed system, which correctly identifies the optimal model in about 91% of the synthetic datasets and 87% of the datasets of real-world applications.

This approach is very accurate, but it is time-consuming to build the large meta-dataset, and relies heavily on historical training examples.

4. DivBO: Diversity-aware CASH for Ensemble Learning (2023)

Combined Algorithm Selection and Hyperparameter Optimization (CASH) is a problem addressed in AutoML in the paper “DivBO: Diversity-aware CASH for Ensemble Learning” (2023). The authors suggest a DABO framework based on a Bayesian optimization procedure, which accounts for the performance and diversity of models.

Most of the existing AutoML frameworks aim to identify the best model configuration. But ensemble learning is better done with the use of several different models. The DivBO framework proposes a diversity surrogate model that simulates the diversity among candidate models and incorporates it into the optimization process.

Multiple experiments demonstrate that DivBO is able to create more accurate ensemble models than current AutoML methods. But the optimization process still needs considerable computation time because of the vast search space.

5. Context-Aware Automated Feature Engineering (CAAFE) (2023)

The study “Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering” (2023) by Hollmann et al. proposes a novel approach that integrates LLMs into feature engineering processes. The proposed system is based on a Large Language Model (LLM) that will be responsible for generating Python code which will be used to implement new features based on the description of the datasets, and the context.

The framework is iterative, with the LLM suggesting the transformations of the features, followed by assessing these changes with machine learning models. The new features are kept in the dataset if they are helpful for prediction. Experimental results show that the system performs better in the prediction task on different datasets.

Although this approach worked well, it is dependent on the description of the textual dataset and may not be effective if there is not enough contextual information.

6. AutoML-GPT: Automatic Machine Learning with GPT (2023)

The paper “AutoML-GPT: Automatic Machine Learning with GPT” (2023) presents the automated machine learning system using GPT-based large language model. The authors suggest a context in which GPT models create prompts that define data sets and machine learning tasks. These are some of the prompts that enable the automation of different components of the machine learning workflow, such as data preprocessing, model selection and hyperparameter tuning.

The system automatically creates machine learning pipelines from natural language descriptions of datasets. The experimental results show that AutoML-GPT can effectively automate complex machine learning processes in various fields.

The method, however, is heavily reliant on prompt engineering and the reasoning capabilities of LLM, and it might show inconsistent performance.

7. The Future of AutoML: How you can contribute (2024)

This survey titled “AutoML in the Age of Large Language Models: Current Challenges, Future Opportunities and Risks” (2024) looks into the crossroads of AutoML and LLM technologies. The authors explore the potential of LLMs to enhance AutoML systems, offering contextual reasoning, natural language interaction, and domain knowledge integration.

The survey also identifies some potential challenges, including the computational cost, lack of transparency, and reliability issues with LLM-based systems. The authors note that embedding LLMs into AutoML systems has the potential to revolutionize the way machine learning is done in the future.

8. MLCopilot: Harnessing the Ability of Large Language Models to Accomplish Machine Learning Tasks (2024)

The paper “MLCopilot” (2024) suggests a framework for supporting the design of machine learning solutions with the help of LLMs. Based on the characteristics of the datasets and the task requirements, the system analyzes them with an LLM, and generates machine learning pipelines.

Unlike the traditional AutoML frameworks, which are based on search-based optimization, MLCopilot tries to emulate human data scientists' reasoning process. The LLM recognizes past machine learning experiences and uses the same approach to solve new problems. This system can generate comparable machine learning solutions as shown in experimental results.

9. The Workshop for FairSense: Long-Term Fairness Analysis of ML Systems 2024 (2024)

A recent paper titled “FairSense: Long-Term Fairness Analysis of ML-Enabled Systems” (2024) examines and assesses fairness in ML systems. The authors suggest a methodology based on simulation that examines the interaction of machine learning decisions and environment over time.

The study notes that the environments in which machine learning systems operate can cause unfairness, due to feedback loops between the environments and the systems. The analysis of various system configurations and identification of potential fairness risks in the FairSense is done through Monte Carlo simulations.

10. EDCA – An Evolutionary Data-Centric AutoML Framework (2025)

This paper “EDCA – An Evolutionary Data-Centric AutoML Framework” (2025) introduces a data-centric automated machine learning (AutoML) framework. The majority of the traditional AutoML frameworks are based on model selection and hyperparameter optimization. EDCA, however, focuses on data quality enhancement by pre-processing data, including data cleaning, data reduction, and data transformation.

The framework employs evolutionary algorithms for searching the space of machine learning pipelines for optimum results at reduced computation costs. The experimental results demonstrate that EDCA is able to outperform the competing methods with less training data.

11. Debugging and Repairing AutoML Pipelines (2025) (11th)

The paper “DREAM: Debugging and Repairing AutoML Pipelines” (2025) tackles the inefficiency of AutoML systems in searching the space of pipelines and suboptimal pipeline configurations. The authors suggest a framework for tracking AutoML pipelines and automatically detecting problems that affect the performance of the system.

The DREAM system expands search spaces and adds optimization techniques based on feedback to enhance the efficiency of the pipeline. The framework has been shown to enhance search efficiency and the accuracy of the models.

12. LLM Meeting Decision Trees on Tabular Data (DeLTa) (2025)

The paper "LLM Meeting Decision Trees on Tabular Data" (2025) presents the DeLTa approach that combines LLM reasoning with decision tree models for predicting tabular data. The system is not directly based on tabular data, but instead relies on decision tree rules as intermediaries to process the data for use by the LLM.

The method makes it possible to have LLMs evaluate logical decision rules and suggest changes to rectify prediction errors. Experimental results show that the model is more accurate without sacrificing model interpretability.

13. Auto PDL: Automatic prompt optimization for LLM (2025)

Solves the problem of creating prompts for large language models. The authors define prompt optimization as an AutoML task and use successive halving algorithms to optimize prompt configurations.

The system compares and contrasts various prompt structures with various LLM models and tasks to automatically determine which prompt structure works best. The experimental results reveal good enhancements in model performance.

14. Survey:Meta learning for automated algorithm selection and parameterization (2025)

The survey "Meta-Learning for Automated Algorithm Selection and Parameterization" (2025) offers a comprehensive survey of meta-learning approaches for algorithm selection and hyperparameter tuning. The study emphasizes the significance of data meta-features in order to predict the performance of algorithms and reviews different meta-learning frameworks that have been developed over the past few years.

The authors also highlight the problems of scalability and computation of existing meta-learning methods.

15. coFEH: LLM-driven Feature Engineering with Hyperparameter Optimization (2026)

The paper “CoFEH: LLM-driven Feature Engineering Empowered by Collaborative Bayesian Hyperparameter Optimization” (2026) introduces a framework which combines the use of LLM-based feature engineering with the use of the collaborative Bayesian hyperparameter optimization. The system dynamically coordinates the feature engineering and hyperparameter optimization processes with a collaborative optimization strategy.

Experimental results demonstrate that the proposed approach enables the improvement of the machine learning pipeline performance by jointly optimizing feature transformations and model parameters.

TABLE 1: INTEGRATED SUMMARY

Year	Paper	Method	Key Idea	Strength	Limitation
[1]2021	Algorithm Selection on a Meta Level	Metalearning ensemble	Combines multiple algorithm selectors	Improves prediction accuracy	Requires large training datasets

[2]2022	MANAS	Repository-based NAS	Uses GitHub models as starting points	Faster model search	Limited to deep learning architectures
[3]2022	Best Model for My Data	Meta-learning	Predicts best model using dataset meta-features	High prediction accuracy	Requires pre-built meta-dataset
[4]2023	DivBO	Bayesian optimization	Diversity-aware model selection	Improves ensemble performance	High computational cost
[5]2023	CAAFE	LLM-based feature engineering	Generates new features using LLM	Improves feature quality	Dependent on dataset descriptions
[6]2023	AutoML-GPT	LLM automation	Uses GPT for ML pipeline automation	Reduces manual effort	Prompt design complexity
[7]2024	AutoML + LLM Survey	Conceptual analysis	Explores integration of LLM and AutoML	Identifies future directions	Mostly theoretical
[8]2024	MLCopilot	LLM reasoning	Generates ML solutions using reasoning	Reduces development effort	Limited scalability
[9]2024	FairSense	Simulation framework	Evaluates fairness in ML systems	Addresses long-term bias	Not focused on model selection
[10]2025	EDCA	Data-centric AutoML	Optimizes data preprocessing	Energy-efficient ML pipelines	Requires evolutionary search

[11]2025	DREAM	Pipeline debugging	Repairs inefficient AutoML pipelines	Improves search effectiveness	Monitoring overhead
[12]2025	DeLTa	LLM + Decision Trees	Enhances tabular prediction	Maintains interpretability	Focused on tabular datasets
[13]2025	AutoPDL	Prompt optimization	AutoML-based prompt tuning	Improves LLM performance	Requires large prompt search
[14]2025	Meta-learning survey	Survey study	Reviews algorithm selection methods	Comprehensive overview	No new algorithm
[15]2026	CoFEH	LLM + Bayesian optimization	Joint feature engineering and HPO	Improves ML pipeline performance	Computational complexity

2.2 IDENTIFIED RESEARCH GAPS

While there has been substantial work around Automated Machine Learning (AutoML), meta-learning and Large Language Model (LLM) assisted machine learning systems, there are key challenges that still exist. Based on the literature review the following research gaps were identified.

1. Lack of Context aware Model prediction

The first is the absence of context-aware model prediction. The first is the lack of context-aware model prediction. The current AutoML systems use search-based methods like grid search, Bayesian optimization and evolutionary algorithms to find appropriate models. These methods can yield powerful models, but demand a lot of computing power and iterations. The current systems do not make use of the context of the datasets to predict the suitable model directly.

2. Heavy dependence on computationally expensive search.

Some AutoML systems, like hyperparameter optimization and Neural Architecture Search, need to involve large search spaces and repeated model training. This results in the cost of computation and time needed to execute the programs are very high and might not be feasible in practice when used in real-world applications with limited computing resources

3. Use of Large Language Models with limited integration into Machine Learning Pipelines

There has been a recent trend of investigating how to apply LLMs to feature engineering, prompt optimization and generation of pipelines. The use of LLM reasoning to predict machine learning models based on the context of a dataset, however, is still not widely adopted. The focus of the existing approaches is primarily on code generation or feature transformation and not model prediction.

4.The dataset's meta-features and semantic context were not fully utilized.

The idea of meta-learning is to try and learn what the best algorithm is based on the statistical characteristics of the dataset. Other approaches, however, tend to only consider numeric features of the dataset and neglect to take into account the semantic or contextual aspects of the dataset, such as the domain knowledge, feature meaning or task description.

5. Failure to locate an appropriate model for recommendation

Inability to find a suitable model to recommend. Existing methods offer static model recommendations using pre-trained meta-models. Such systems do not have any capabilities to adaptively learn and update the model recommendations as new data or context is introduced..

6. Limited explanation ability in Automated Model Selection

Many AutoML systems are black-box tools, which will make it hard to comprehend what the model was chosen for. The need for systems that allow for understanding of the reasoning behind model predictions, particularly in critical applications like healthcare and finance.

7. Efficient and scalable model selection frameworks are needed.

The current solutions lack in scalability, especially with large datasets or dense feature spaces. As a result, a scalable model selection system which integrates contextual reasoning and efficient machine learning evaluation methods is needed.

From these gaps, it is evident that there is a considerable need to create a context-aware intelligent system that can support the prediction of appropriate machine learning models with the reasoning power of Large Language Models.

2.3 OBJECTIVE OF THIS WORK

The key objective of the proposed research work is to create a system called Context-Oriented ML Model Prediction and Adaptive Selection System based on Large Language Models that will efficiently analyze the characteristics of datasets and suggest the best machine learning models.

The list of research objectives that will help reach the above-stated goal is as follows:

1. Dataset Characteristics Analysis

Conducting analysis concerning the properties of a dataset: its size, type of features, missing data, distribution, and other aspects.

2. Meta-Features Extraction

Deriving meaningful meta-features of the dataset that may be used for characterizing it and making predictions regarding the suitability of a machine learning algorithm.

3. Contextual Understanding of Datasets through Large Language Models

Using large language models and their reasoning capabilities to interpret datasets.

4. Intelligent Model Prediction

Development of a system that will help predict which machine learning algorithms are most appropriate for the dataset in question without applying complex search mechanisms.

5. Adaptive Model Selection Process

To design an adaptive process that selects and recommends appropriate machine learning algorithms depending on the features of the dataset and other contextual analyses.

6. Integrating with Machine Learning Evaluation Techniques

To incorporate the new model prediction technique with the machine learning frameworks in order to train and test the recommended machine learning models.

7. Performance Measurement

To assess the performance of the suggested technique through benchmarking with other machine learning algorithms using evaluation measures like accuracy, precision, recall, and F1 score.

8. Enhancing the Efficiency of Model Selection

To minimize the computational burden involved in conventional autoML techniques through the incorporation of context-driven reasoning during model selection.

CHAPTER 3

ANALYSIS, DESIGN AND MODELLING

Machine learning has become an important part of the development of software systems and data-centric applications. Machine learning techniques have become ubiquitous for automation of predictions and making decisions on top of data analysis in domains like medicine, finance, cyber security, and recommendation engines. However, despite the remarkable advances in machine learning, developing a machine-learning system remains quite a challenging task.

The standard way that these algorithms are trained involves extensive experimentation through trial and error. The developer repeats this process a number of times, involving not only the preprocessing of the data but also using multiple algorithms, adjusting the hyper-parameters, and evaluating the results from this analysis. This process takes a considerable amount of time and computing power. Experience and technical skills are also needed for machine learning processes.

Automated Machine Learning (AutoML) was devised in order to help in automating some functions, such as selecting an appropriate algorithm and optimizing its parameters among other functions. Current AutoML frameworks generally make use of exhaustive search optimizations. This method is tedious and difficult to perform manually while using AutoML algorithms, which increases the time required to do so depending on the number of model combinations that must be tested.

In the presented project, the Context-Oriented ML Model Prediction and Adaptive Selection System for Large Language Models (LLM) is developed. It incorporates the metadata analysis of datasets, context-based reasoning, as well as adaptation to the process of machine learning models' training and learning to improve model selection.

Unlike simply doing lots of random trials, the proposed framework tries to intelligently reduce the amount of candidates to search for by analyzing the characteristics of the target dataset prior to starting the model training process. This unified system combines AI-powered reasoning, machine learning-driven workflows, and automated processes, all while ensuring the implementation of AI principles to boost efficiency and reduce computational costs to enhance the explainability of recommendations.

The general architecture and design of the proposed framework are described in this chapter. It provides insight into how the system deals with data sets, how it pulls out metadata, which it uses in making recommendations, how it automates the training of the system and how it stores knowledge/insights so that they can be used on an ongoing basis to continually improve. In addition, the work-flow and the main components of the implementation of the proposed intelligent recommendation system are presented in this chapter.

3.2 PROPOSED SYSTEM OVERVIEW

The proposed system is an intelligent machine learning based recommendation system and adaptive selection system by considering the context in order to ease and to automate the machine learning selection process. This architecture seamlessly integrates machine learning automation with the reasoning power of Large Language Models (LLMs), resulting in a more efficient and intuitive recommendation system.

The first goal of the system is to examine the characteristics of the datasets, and make a recommendation for an appropriate machine learning model at the start of a large amount of experimenting. Unlike AutoML frameworks that are originally based on methods of optimization through an exhaustive search, the proposed approach aims at minimizing unwarranted calculations and increasing the quality of the recommendations based on the contextualization of the data and smart reasoning.

The model system suggested in this study starts with the upload of the datasets. Structured dataset can be uploaded by the users via frontend interface in CSV format. After the data is uploaded the backend system assigned will process the data and extract the key metadata information like Number of rows, columns, feature Types, missing values, statistical summaries, target variable info etc.

Once extracted, the information in the metadata file can be used by the user in order to choose the target column to be used for the prediction problem. Users can also remove unwanted features from, or columns in, the data set prior to making a recommendation. This flexibility enables the users to tailor the workflow as per their needs and requirements.

The information pulled out of the metadata, sample data records, and (optional) user instructions are then sent to the LLM. The LLM serves as a smart recommendation system

which can analyse the contextual information and make appropriate machine learning model recommendations. The system considers more models to test; however, it does not test every model in the system, it only considers the model that is more likely to be effective for the specific data.

In particular, a key reinforcement of the proposed framework is the provision of explanations for the recommended steps. The LLM can provide reasoning and explanations for the appropriateness of specific algorithms for given datasets, in addition to the model recommendations. This provides for greater transparency and enhances user's understanding of the recommendation process.

Once recommendations are made, the system will automatically start the model training pipeline. Machine learning models are recommended and trained using machine learning libraries like Scikit-learn, XGBoost. The framework checks the accuracy, precision, recall, F1 score, ROC-AUC etc. of each trained model accordingly to the type of problem.

The system then decides on which of the models is the best and produces artifacts that can be downloaded and evaluation reports. The outcome of these results are shown via the dashboard interface, allowing users to visualise performance indicators, as well as compare forecast models.

In total, the framework is designed to develop a realistic, intelligent, adaptive and explainable machine learning recommendation system which integrates the power of LLM based machine learning and automated machine learning techniques. To make machine learning workflows easy and quick, to reduce manual tasks, to enhance the quality of the recommendation and to enable effective decision making for both technical and non-technical users.

3.3 SYSTEM ARCHITECTURE

The proposed work on the Context-Oriented ML Model Prediction and Adaptive Selection System based on LLM is envisioned as a cohesive intelligent system that integrates analysis and understanding of the datasets, manages the contextual problem by invoking relevant problem-specific LLMs, automates the machine learning workflows and implements the adaptive learning systems. Its architecture is modular, enabling various components to function

effectively and collaborate seamlessly with each other, while also allowing for scalability and flexibility for future enhancements and expansion.

The system is primarily designed in five major user-mapped layers, which are: User interaction layer, Dataset processing layer, Recommendation layer, Machine learning execution layer, Knowledge base layer. The function of each layer in the overall workflow of the system.

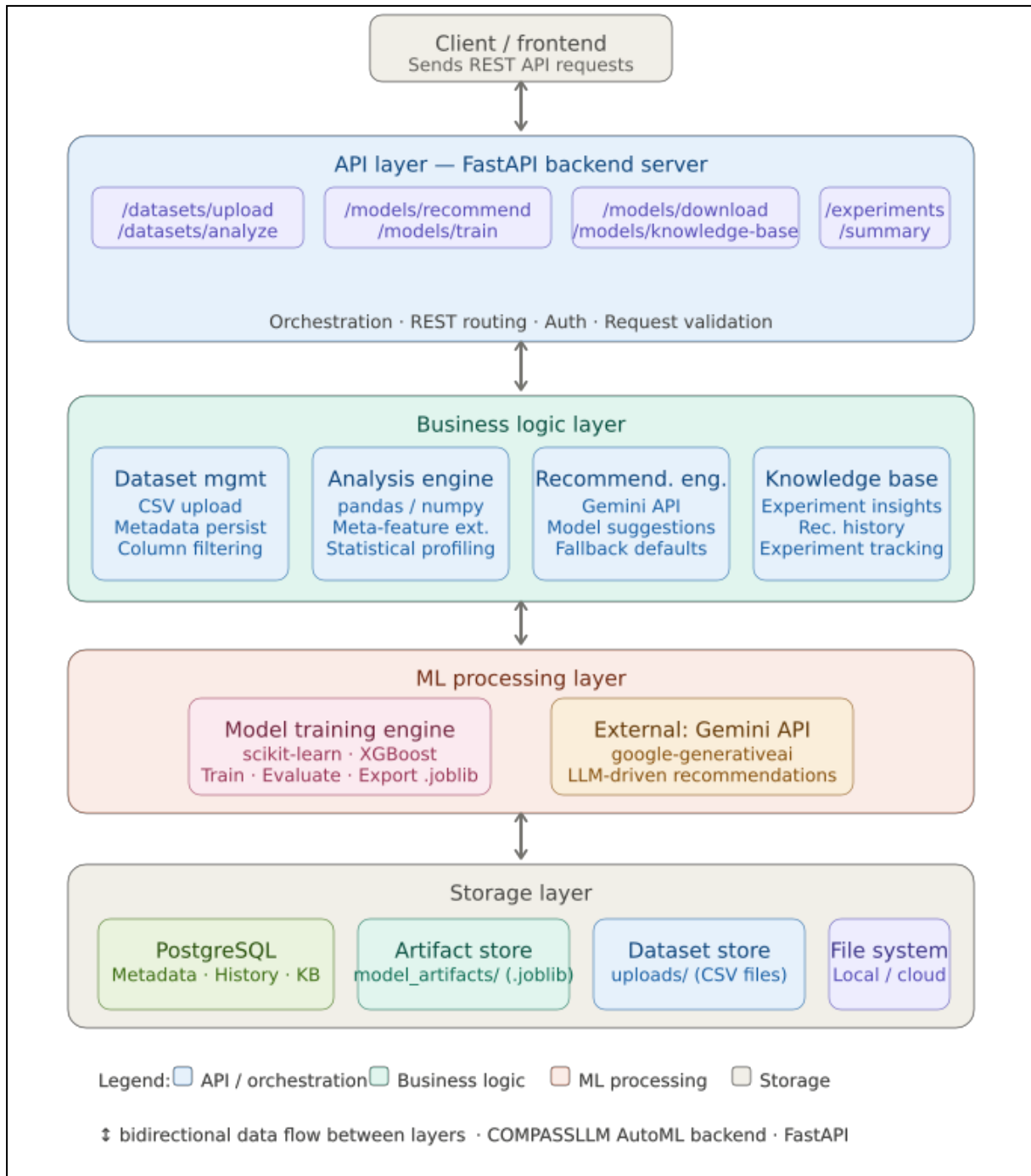


Fig 1. Architectural Diagram

The first layer is the user interaction layer or "frontend" layer of the framework. The layer is designed for user to communicate to the system. Users can upload datasets, choose data that they want to model, filter data that they don't want to model, recommend on the data, see the evaluation results, download the trained model artifacts from the interface, etc. Developed using React, TypeScript to deliver an interactive and responsive user experience.

The second layer is the Layer of the data set processing and the extraction of the metadata. Upon uploading the data file, a slight analysis is done on the dataset on a pre-processing level by the backend. Highlights relevant metadata, including:

The amount of rows and columns,

- feature types,
- missing values,
- statistical summaries,
- data distribution,
- Multiple categorical and numerical feature information,
- The characteristics of and and the target.

All of the metadata gives the context of this dataset and is a crucial input to the recommendations process. The information extracted is temporarily stored and ready to utilize for further analysis.

Explainability is another desired functionality of this layer. The LLM also provides reasoning that is understandable to humans for explanatory recommendations on which algorithms may work for the dataset. This will help increase transparency and aid in user understanding of the recommendations process.

The fourth layer is a layer of execution and evaluation of a machine learning machine. Once recommendations are created, the back end automatically starts training the recommended machine learning models like Scikit-learn and XGBoost. The preprocessing, training and validation and evaluation workflows are automatically handled in the system.

Performance metrics are used to assess the performance capabilities of each of them with appropriate quantitative indices for the prediction task. Various metrics such as ROC-AUC,

accuracy, precision, recall, F1 and R^2 score are appropriate for classification problems, and for regression problems, Mean Squared Error and R^2 score are appropriate.

After evaluation, the optimal model is identified and downloadable model artefacts are created as well as evaluation reports. Results are presented in the dashboard interface and user analytics and comparisons are provided.

The knowledge base and adaptive learning layer are the fifth layer. It is an element to store the recommendation results, training results, the measurement of evaluations, setting by the user, and experimental data (historical data) from the experiment. The use of this layer would be to facilitate ongoing learning and learning adaptation.

The framework captures historical data and can be used going forward to make better educated recommendations and be more efficient in decision-making. This way, you will make it become an intelligent system (dynamic recommendation system).

This is achieved through FastAPI, which handles all communication between the API, processing of all the datasets, requests for recommendations, and training operations in the back-end. There's one database, known as PostgreSQL, that handles metadata, recommendations and historical data.

As a whole, the architecture for the proposed system aims to develop an intelligent, scalable, adaptive, and explainable machine learning recommendation system that could simplify the machine learning process and alleviate computational complexity.

3.4 WORKFLOW OF PROPOSED SYSTEM

The proposed system's workflow is designed to simplify and automate the process of selection of machine learning models with the help of context-based reasoning and adaptive machine learning techniques. It merges the power of data analysis, Large Language Model suggestions, auto training, assessment, and ongoing learning seamlessly into a single workflow.

The data flow starts by uploading the data set. Structured data is uploaded by the user on the frontend whose data is in the form of CSV representations. After upload of data set, the back end securely keeps that data set and begins the data set processing operations.

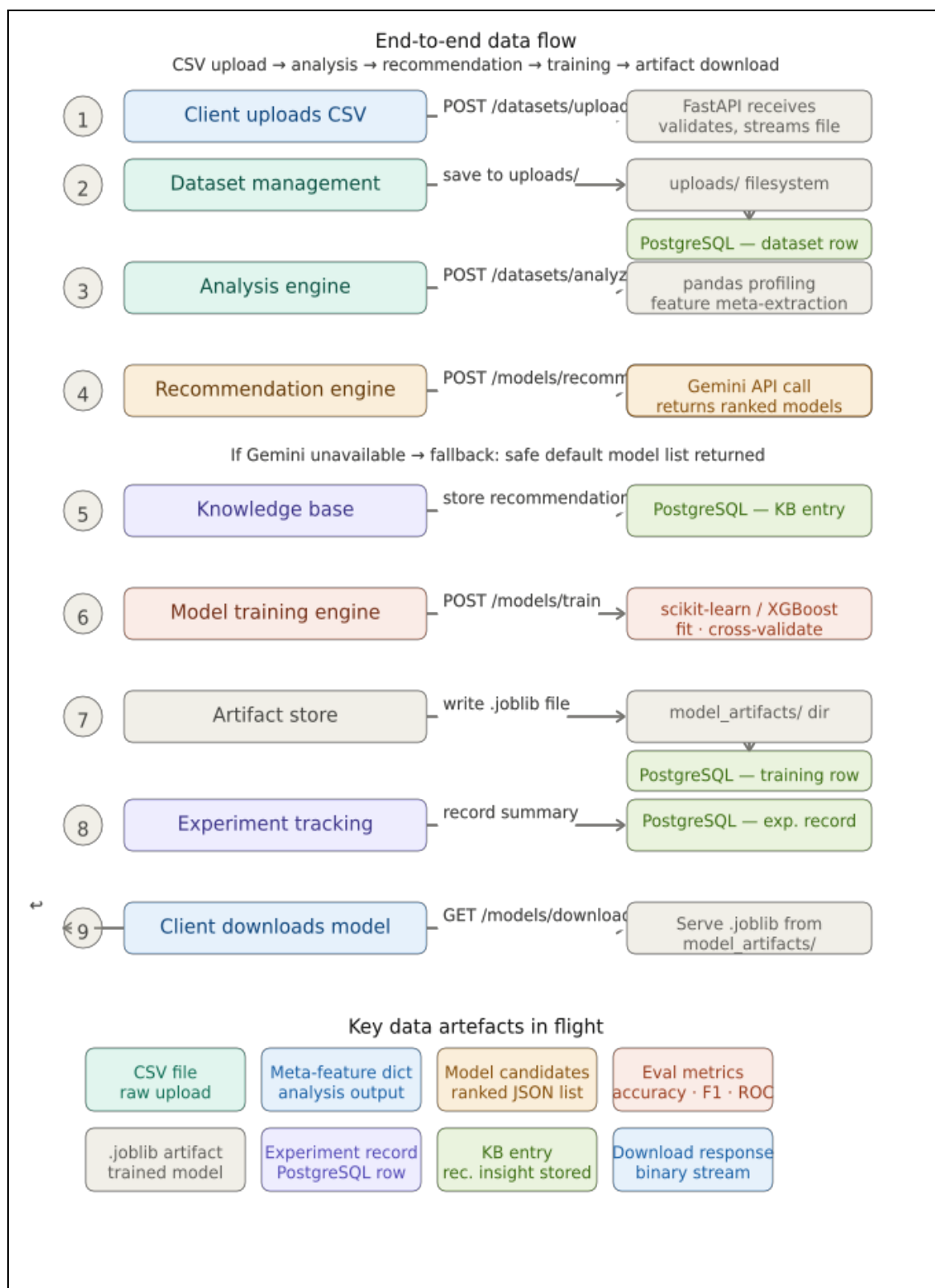


Fig 2. End to End Flow

The next step is to extract metadata and analyze data in the datasets. The values of the input data set are retrieved to get important data including number of rows, number of columns, data type of features, number of missing values, statistical data, categorical features, numerical distributions etc. This step will facilitate the system of its understanding of the structure and complexity of uploaded data.

Once metadata is extracted user can choose a target column which is the prediction objective. This is done by selecting the target columns which will help the system to determine the type of the task (classification/regression). Users can also use pre-processing steps to remove the irrelevant columns/_features from the data-set before the model training could start.

After the metadata and data structure of the dataset are determined, the data and properties extracted from the sample data are given to the Large Language Model. This can also include the user instructions or constraints that are optional. The LLM takes in piece of contextual details and produces appropriate machine learning model suggestions as outlined by data characteristics.

The proposed system is quite different than the traditional AutoML process where a large number of algorithms are tested, without having any knowledge of the context. Depending on the properties of the dataset, different algorithms are suggested to be used in LLM assignments, including Random Forest, XGBoost, Support Vector Machine, Logistic Regression or Neural Networks.

A crucial characteristic of the workflow is explained recommendations. The LLM is also capable of giving reasoning regarding the suitability of certain algorithms for the data set, along with suggestions, this reasoning is provided by the LLM. This makes it easier for users to understand the rationale behind the recommendation, and also creates transparency.

Once recommendations are made, the system automatically starts training the model pipeline. Machine learning models are recommended, and they are trained with the machine learning libraries embedded in the machine learning back end. The framework automatically does all of the pre-processing, model-fitting, validation and evaluation during this process.

The accuracy, precision, recall, F1 score, ROC-AUC or regression evaluation metrics are used to check the accuracy of the trained models with appropriate performance metrics, depending

on the problem type. The system will test and verify the performances of the models and select the most successful algorithm.

The framework produces model artefacts and evaluation reports once the evaluation is finished. Trained models can be downloaded, and performance results displayed in the dashboard interface. Even training summaries, outcomes of recommendations and training history can be managed on dashboard for analysis.

One of the critical steps in the process is updating the knowledge base. The system stores the results of the recommendations, the performance of training and the evaluation metrics within the database, alongside with the configuration details. This historical data becomes later on used to further enhance future recommendations and adaptive learning.

With the continuous accumulation of the experimental results and recommendations information of the system, its workflow becomes more intelligent over time. This constant learning ability allows the framework to slowly grow and evolve the quality of the suggestions in various datasets and areas of interest.

The anticipated workflow is expected to minimize the manual work, amount of unnecessary experimentation, explanation of the recommendations and ease of machine learning building for technical and non-technical users. The combination of Large Language Models and machine learning automation brings a more intelligent and context-aware recommendation system significantly better than traditional AutoML approaches.

3.5 DATASET UPLOAD AND METADATA EXTRACTION

The first phase of proposed system workflow is to upload the datasets and extract the metadata. This step is crucial as it can directly impact the performance of the machine learning models and the accuracy of their recommendations. The system needs to first analyze the properties of uploaded database before any operations (recommendation, training, etc.) is done.

The proposed system will enable submitting of structured data completely accessible via frontend interface in csv format. CSV files are very common e.g. in machine learning workflows, as they are simple, light-weights and easy to process across platforms and programming environments. The front-end interface will be designed to give an easy and user

friendly upload system to facilitate easy interaction with the system even for those users with limited technical skills.

After uploading, the back-end system will process the uploaded file, which will start the preprocessing and analysis process. Securely stored upload file securely up to the time the framework starts extraction from the dataset and extracts few important info. In this stage, the system verifies the uploaded file ensuring that the data-set serves the correct format and is readable. Should any file corruption, unsupported files or other data structures exist that are incorrect, the system will produce correct error messages for the user.

Once the framework was successfully validated, it conducts the metadata extraction process. Metadata is the information which describes data and is used for understanding the structure and complexity of this data before machine learning operations such as data analysis or machine learning models are created. The proposed system consists of analyzing the dataset context in first phase by utilizing metadata analysis, and then, directly training multiple models in the second phase without doing so blindly.

During the metadata extraction process the following is identified:

- the number of rows, the number of columns
- feature names
- data types
- Both categorized and quantitative attributes
- missing values
- statistical summaries
- duplicate records
- Histogram of target variable distribution and distribution of the variable with response variable.

When dealing with numeric properties, it determines statistical properties like the average, median, standard deviation, minimum, maximum and quartile distributions. These summaries inform of patterns, outliers and spread of data from the data set.

For discrete features, it examines rows of values, category frequency and distribution of categories. This is significant when suggesting machine learning algorithms since some algorithms are capable of coping with categorical data better than others.

This structure also determines the missing values in the data set. A key challenge arises in machine learning tasks that are encountered in real life, where the information is not always available, and the impact these missing pieces can have on the performance of the model can be significant. The system uses this information to identify missing data at the metadata extraction phase, thus obtaining more context in the dataset for appropriate preprocessing and/or good model selection.

Another crucial operation carried out during the metadata extraction phase, is the identification of the type of the datasets. The system will classify the problem based on the distributions of target variable and features selected. This further reduces the universe of machine learning algorithms that end-users can be recommended in this framework.

Then from wherever the metadata is found, the extracted metadata is temporarily stored in the back end and subsequently utilized in the electronics recommendation process. The framework does not only process the dataset as numerical data but also tries to construct an understanding of the structure of the dataset from a contextual perspective. This context understanding will be useful as an input for the Large Language Model to generate the recommendations.

Consistently identifying metadata in the experimental process is a crucial element to decreasing redundant experimentation. Traditional AutoML systems typically start training a model without any in-depth knowledge of the data. The proposed framework introduces a preliminary intelligence layer via metadata analysis to enable a better recommendation process, compared to the single layer approach of using acclaim metrics.

The other benefit to extracting metadata is that it improves explainability. Actually, the system can handle the characteristics of the datasets, which enables the generated recommendations to reason using the real properties of the dataset. This will render the recommendation process clearer and much simpler to use.

The dataset uploading and metadata stage in which metadata is extracted is considered as a critical component of the proposed framework. It allows the system to perform intelligent

analysis of the dataset structure for intelligent contextual information to build up for generating recommendations, and enhance machine learning workflows.

3.6 TARGET SELECTION AND FEATURE EXCLUSION

In this paper, aspects of the proposed machine learning recommendation workflow, such as target selection and feature exclusion, play an important part. These steps enable users to set up the dataset beforehand, and direct the system to utilize the important aspects of the dataset that are essential for predictive tasks.

Once all the metadata is extracted, the information about the uploaded dataset is displayed on the front end interface of the system. The user then needs to choose a column that is used as the output variable – the machine learning model will try to predict this column.

A crucial step in the process is the selection of the targets - it defines the type of machine learning problem. The system classifies a given target variable to determine if it is a classification or a regression. For instance, if the target variable has a dimensional output of categorical representation (Yes/No – labeling) the framework assumes the problem is a classification problem. Likewise, when the target variable values is a numeric value (continuous) the task is indicated as regression.

The correct target selection is key to create appropriate models recommendations. Each machine learning algorithm is suited to fuel other types of prediction problems. Usually, for categorical prediction problems, classification algorithms like Logistic Regression, Random Forest Classifier, Support Vector Machine and XGBoost Classifier are used, and for continuous numerical prediction problems, regression algorithms like Linear Regression, Random Forest Regressor, and Gradient Boosting Regressor are used.

The interactive interface designed in the proposed system makes it possible for the selection of the desired variable to be an easy process. This makes the model more user friendly and allows users to control the model learning process.

The other major feature that is offered by the framework is feature exclusion. In the real world, there may be irrelevant, duplicate, noisy or unnecessary columns that could have a negative impact on the performance of the machine learning. There may be values/constant values that

do not relate to prediction that can include identifiers, times, or other information that are not particularly useful.

The feature exclusion module is used to exclude these columns from the data a user does not want to use when training the data set. Users can exclude the unwanted features from the image in the front end user interface, and the new configuration will be sent back to a system that can be used as a backend system for further processing.

Excluding features can benefit machine learning workflow in several ways. Taking out the unnecessary features can:

- reduce dataset dimensionality,
- improve training speed,
- decrease computational complexity,
- minimize noise,
- Improve the generalisation of models and enhance their performance

For instance, fields like user ID or serial number typically when defining an identifier might not provide much helpful predictive information. The presence of such features can result in model over-fitting and/or model complexity. Likewise, features that have high correlation or are redundant can have an adverse impact on some algorithms.

The proposed system enables the users to redo the analysis if changes their feature configurations. The advantage of this is that feature combinations for that data set can be varied without republishing the data set. Feature exclusion is tested after the metadata and/or dataset summary are created to keep the recommendations contextually accurate.

This module offers another significant benefit in terms of the quality of recommendations, which also plays a vital role in the module's added value. The metadata is maintained with the updated information, which enhances the context-awareness of the Large Language Model (LLM) used for recommendations and improves the structure of the data set.

Target selection and feature exclusion also helps towards the explainability and user control. The framework is not a black box type system; users have the opportunity to play an active part in the decisions they make regarding the design of the datasets they will use and in their preparatory mechanisms.

In general, this module enhances flexibility in workflow, quality of the data, computational efficiency, and accuracy in recommendations. The proposed framework not only leverages the user interaction but also incorporates automated metadata analysis for a more practical and intelligent recommendation process by machine learning.

3.7 LLM-BASED RECOMMENDATION ENGINE

The intelligent part of the proposed intelligent system is the LLM-Based Recommendation Engine. It serves the main purpose of analysing the attributes of datasets to recommend appropriate machine learning model(s) through contextual reasoning. The proposed model integrates semantic understanding and reasoning into the recommendation process by leveraging LLM models, while traditional AutoML systems are primarily based on exhaustive trial and error optimization and numerical values.

Once extracted and dataset preprocessed, essential data relevant to the various datasets, including feature types, size of each dataset, missing values, statistical summaries, information about the target data variable, and each sample in the dataset is passed on to the Large Language Model. The user instructions can also be optional, to give additional context to the prediction task.

Based on this information, the Large Language Model uses machine learning techniques to try to decide which machines learned algorithms might be effective for their data set. The recommendation process is intended to mimic the way 'first-class citizens' of the data science field arrive at recommendations: some algorithms work better on other types of datasets that the data expert has learned to trust, and some only work when used on a 'regional' scale, while a 'local' scale might well be enough.

For instance, if the dataset is structured, and provided that it has moderate class imbalance and categorical features, then algorithms of the tree type will outperform others like Random Forest or XGBoost. Likewise, LR or SVM can work well on smaller data sets which have less complex relationships or are more linear. The LLM reads these patterns in the context and makes recommendations based on that.

Another important functionality of the recommendation engine is explainability. This is followed by algorithm suggestions and explanations of the nature of the possibly applicable

algorithms in terms of the text, that is to say human-readable explanations. This increases the transparency and clarifies the users on how the recommendations are made.

Recommendation engine also helps in pruning away futile computation. Rather than evaluating all the machine learning algorithms blindly, the system automatically focuses on more relevant context-driven algorithms in its search. This helps enhance the efficiency and shorter execution time during model training.

In general, the LLMBased Recommendation Engine is a significant enhancement of the framework from a traditional search-based AutoML system into a more adaptable, context-aware and intelligent recommendation system.

3.8 PROMPT ENGINEERING STRATEGY

Prompt engineering is an essential aspect of the performance of the recommended framework. As LLM's rely heavily on the prompt given and the context information provided to them, it is crucial to create the prompt carefully if you want to get accurate and meaningful suggestions.

The proposed system dynamically generates prompts based on input given by the user and metadata from the extracted data sets. The prompt contains important information such as:

- dataset dimensions,
- feature types,
- missing values,
- target variable details,
- statistical summaries,
- Data related to sample dataset records.

This background information enables the Large Language Model to grasp the framework and intricacy of the data set prior to providing recommendations.

The objective of the prompt engineering approach is to encourage the LLM to make meaningful, pragmatic, and low-compute machine learning-based suggestions. The framework eschews for general, context-agnostic instructions and offers structured engineering and contextual instructions specifically targeted to the machine learning model selection task.

Generating the prompt was the other consideration in the prompt design; it had to be clear and consistent. The chances of giving vague or incomplete prompts which lead to inaccurate or inconsistent recommendations. As a result, the system creates the prompts in a consistent format, which enhances consistency and the quality of recommendations.

An optional User Guide is also covered by the framework. Other constraints can be set by the user, for example, it can be required that faster models are used, to reduce computational expense or to prioritise interpretable models. As part of the prompt, these instructions are detailed to make recommendations to suit user needs.

The output from this LLM is then transformed into structured recommendations which can be used on a straight through use in the automated training pipeline.

3.9 AUTOMATED MODEL TRAINING PIPELINE

The machine learning models recommended by the LLM-based recommendation engine are trained, evaluated and compared by the Automated Model Training Pipeline. This module provides real-time modeling and automatic execution of workflows with the machine learning methods, with a significant benefit of avoiding manual experimentation.

Having generated appropriate suggestions by the LLM, the machine learning library (like Scikit-learn or XGBoost) runs the learning process automatically in the back end system. Before training, the system first prepares the dataset based on target variable and feature setup that is selected.

The pipeline is used for carrying out various preprocessing tasks like filling in missing values, encoding of categorical variables, splitting datasets to train and test sets and preparing feature matrices. The pre-processing of data guarantees the obtained dataset is suitable for machine learning algorithms.

Once pre-processing is done, the pre-processing data is processed on individual machine learning models. The framework uses multiple classification / regression algorithms based on the type of prediction problem been figured out during target selection process.

Following the training, the system is tested with the help of suitable performance parameters with each model. In classification tasks values like accuracy, precision, recall, F1 score, ROC

AUC are measured. In regression problems measures like Mean Squared Error, R^2 score can be employed.

The pipeline then evaluates the performance of the algorithms, and selects the best algorithm based on the results. The training summary and evaluation metrics are shown via dashboard interface to aid users in analysing model behavior and evaluation.

Another crucial aspect of the pipeline is artefact production. It creates a model file and result report, which can be downloaded once training is done. Users can deploy for reuse or experiment further with models trained.

The automated training pipeline has achieved a huge reduction in the manual effort by integrating all the preprocessing, training, and evaluation and reporting into a single training workflow. The proposed system combines this pipeline with the LLM-based recommendation engine to form a more intelligent and efficient machine learning system that automatically suggests workouts based on various tasks and metrics. The future system integrates this pipeline with the recommendation engine based on LLM, resulting in a more intelligent and efficient machine learning system that automatically recommends workouts based on various tasks and metrics.

3.10 KNOWLEDGE BASE AND CONTINUOUS LEARNING

The Knowledge base and Continuous Learning module is one of the crucial part of the proposed system as it is able to enhance the frame work recommendations over time. The proposed framework is dynamic, unlike traditional recommender systems which are performed statically but it learns from previous experiments and history. The more information entered and business processes modeled, the smarter and more efficient the system will be as a result of this adaptive behavior.

The knowledge base serves as a central repository for holding key data related to recommendations, model training outcomes, evaluation measures and user setup. Following each training process, system keeps information like:

- recommended algorithms
- dataset characteristics

- selected target variables
- feature configurations
- performance metrics
- execution time
- and best-performing models.

This historical information can be useful for the next recommendation task as it enables the framework to determine patterns where the characteristics of the data sets and the performance of the models are correlated to each other.

If the machine learning algorithms already performing well in these types of datasets have similar structures, for instance, the system can be optimized in future recommendations to preferences for these algorithms. Likewise, the framework can skip testing of the algorithms that, for given data sets, failed to yield good results in the past when a specific type of data sets is encountered repeatedly.

The Auto-Transformer gives the proposed framework an extra advantage of having an innate ability to adapt and learn. The system gradually learns more about the behaviour of the model in various problem domains with increasing number of datasets. This decreases reliance on solely static recommendations and permits it to advance over time with experience gained along the way.

An added benefit of the knowledge base is more consistency in recommendations. Previous experimental results are also stored, so recommendations can be created which are more in line with historical trends of performance. This helps to enhance the quality of the recommendation and reliability of the system.

The knowledge base is also useful for future possibility of the framework scaled up. Later on the framework can be extended further with other learning mechanisms, recommendation optimization strategies and the integration of adaptive feedback systems, without significant changes to the architecture.

Overall, the Knowledge Base and Continuous Learning module make the proposed system something more than just a recommendation system—a system that can evolve to become an adaptive intelligent system that evolves its behavior via experience and historical learning.

3.11 DASHBOARD AND ANALYTICS MODULE

The Dashboard and Analytics Module offers an interactive platform to view machine learning recommendations, training outcomes and evaluation scores. The new module provides a main communication interface with the backend system and enables users to effectively observe the execution of workflows and evaluate the performance of the models.

Once the model has been trained and evaluated, users can then see the outcomes in the dashboard in a well organized and readable format. Key facts like suggested models, assessment scores, training duration and summary, state of execution and comparisons are displayed visually, to make them easier to read and understand.

The dashboard will enable users to check out the performance of different machine learning models with accuracy, precision, recall, F1 score and ROC-AUC score. The "best" algorithms can be more easily discovered by these visual comparisons.

There's also a high degree of workflow clarity on the dashboard. The proposed framework does not work as a 'black-box system' but gives insight into the various steps of the machine learning (ML) pipeline. Users can interactively view the results of the analysis of the selected datasets, the explanation for the received recommendations, the progress of the training process as well as the outputs of the evaluation process.

The dashboard also allows for your trained model artifacts and result reports to be downloaded. This makes it easier to use, and makes it possible to use models produced by the system again to deploy or conduct further experiments.

Taking user experience into consideration, technical users and non-technical users can easily do machine learning more easily on the dashboard. The module organises complex information in a clear and understandable way, thus enhancing accessibility and interacting with the system.

In summary, the addition of the Dashboard and Analytics Module is a vital element in proposed framework to enhance the user experience, transparency of recommendations and interpretation of results.

3.12 DATA FLOW DIAGRAM (DFD)

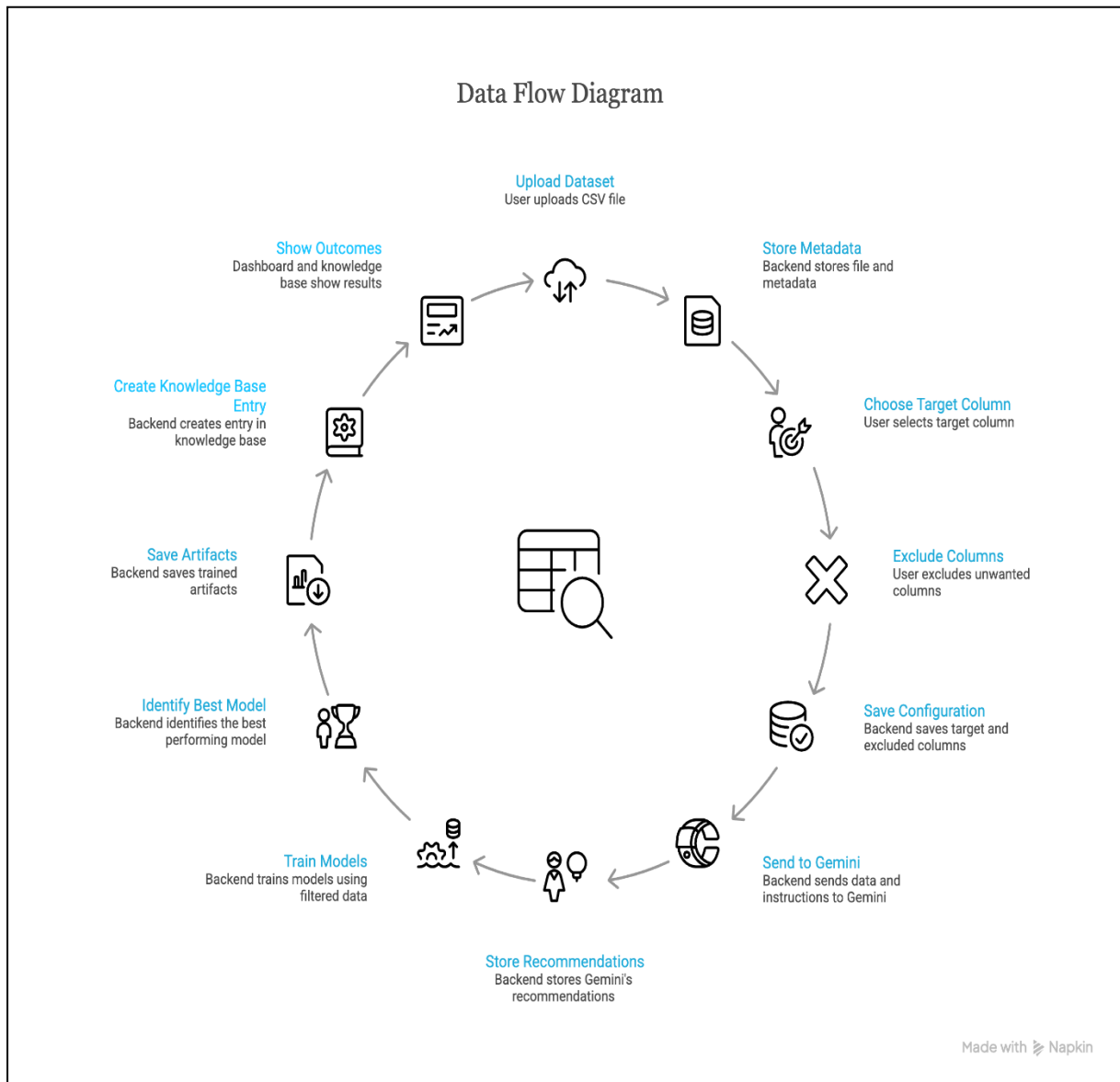


Fig 3. DFD

3.13 FUNCTIONAL REQUIREMENTS

Functional requirements clearly specify the operations the system needs to conduct.

The envisioned system can be expected to have the following functions:

1. Dataset Upload

There will be a facility for a user to upload a data set to analyse.

2. Data Preprocessing

The system should perform some pre-processing operations such as filling missing data and

encoding the categorical data automatically.

3. Dataset Analysis

This system should be capable of examining data characteristics and processing the meta-features.

4. Contextual Model Recommendation

The system needs to employ LLM reasoning to offer/advise appropriate machine-learning solution models.

5. Model Training

The system should be able to train the recommended model on the data.

6. Model Evaluation

The evaluation of the models should be carried out based on a suitable evaluation metrics within the system.

7. Result Visualization

Model performance results and recommendations should be presented to the user by the system.

3.14 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements give an idea of the quality of the system.

1. Scalability

Should accept data sets of varying size.

2. Efficiency

Before your system, the one reason was, it reduces computational overhead to the traditional system.

3. Reliability

The system is supposed to give accurate model recommendations, at every time.

4. Usability

The system interface should be user friendly for both novice and experts.
data scientists.

5. Maintainability

System should be easily upgradable and adapted with newly developed machine learning algorithms.

3.15 ASSUMPTIONS

The assumptions that are made when creating the system are:

- The data sets used for testing purposes are reflective of real-world problems.
- There is enough information contained in the dataset to create suitable machine learning models.
- Large Language Models can efficiently understand the characteristics of the data set.
- The machine learning libraries used for implementation have good training capabilities.

3.16 RISK ANALYSIS

Risk assessment highlights possible risks that can influence the operation of the system.

1. Risk of Poor Data Quality

Data quality problems such as incomplete data and noise could impact the accuracy of predictions by the model.

2. Risk of Heavy Computation

Computation-intensive training of multiple models is required for building a recommendation system.

3. Risk of LLM Availability

External LLM API services pose risks of increased latency and unavailability.

4. Risk of Inaccurate Model Recommendations

The model might make inaccurate recommendations sometimes when the characteristics of datasets are ambiguous.

To address the risks, appropriate data preparation, evaluation methods, and alternative models are considered.

CHAPTER 4

IMPLEMENTATION AND RESULTS

This chapter deals with the process of implementation and experimental realization of the proposed Context-Oriented ML Model Prediction and Adaptive Selection System for Large Language Models. Implementation refers to the stage in the process that entails actualizing the proposed idea and system through which the intelligent recommendation system will be developed for analyzing datasets and making contextual recommendations as well as training, evaluation, and learning models adaptively.

The proposed intelligent system was designed as a fully stacked solution combining both front-end and back-end processing utilizing machine learning techniques and Large Language Model Technology. The main goal is automation of machine learning process and improved contextual recommendations.

The deployment of the solution entailed building several components such as data set upload, metadata extraction, target variable selection, feature removal, recommendation formulation, automatic modeling, analytics, artifact creation, and storage in the knowledge base. The integration of the different components was done in a manner that facilitated effective communication between the frontend, backend, machine learning pipeline, and the database component.

The formulation of a proficient, user-friendly framework capable of streamlining the process of machine learning was one of the goals during implementation. Unlike the manual setup of machine learning processes that entails configuring each individual machine learning technique separately, the proposed approach consolidates all important step into a single intelligent framework.

This solution was also oriented toward avoiding unnecessary experimentation through intelligent suggestions. Instead of blindly picking up any algorithm that could be used to solve the problem, regardless of whether it is appropriate for the given case, the right algorithm is selected beforehand from the analysis of metadata of the data set using Large Language Models.

The entire workflow of the implemented process is as follows:

- dataset uploading & preview,
- metadata extraction & analysis,
- target feature selection,
- feature elimination,
- generation of recommendations using the LLM,
- training of ML models,
- model evaluation & visualization,
- generation of artifacts,
- and maintenance of the knowledge base.

Future adaptability considerations have been put into place, as future features, including adaptive fine tuning as well as many others, will be easily integrated into the project.

This chapter provides more information on how the implementation environment, the frontend implementation, the backend implementation, the database, as well as Gemini API were implemented.

4.2 DEVELOPMENT ENVIRONMENT AND TOOLS USED

The following system implementation involved different modern technologies in terms of frontend development, backend development, databases, and machine learning tasks. In choosing these technologies, scalability, flexibility, ease of integration, and machine learning capabilities have been considered.

For building the frontend part of the application, React and TypeScript were used. React is utilized in the project since it provides a flexible framework that enables state management, uses UI components, and creates interactive user experience. Using TypeScript, one can keep the frontend code organized and easy to maintain.

FastAPI technology was adopted in the implementation of the backend services for the proposed system. FastAPI technology was adopted due to its lightweight nature, fast performance, asynchronous request handling capabilities, and effective support in developing RESTful APIs.

Considering the above criteria, PostgreSQL was selected as the database management system to store:

- metadata of datasets,
- results of recommendations,
- metrics of evaluation,
- training history of previous sessions,
- and knowledge base of the data.

PostgreSQL was chosen due to its high level of efficiency, reliability, and capacity to handle structured data.

- Python libraries are being used for executing machine learning tasks include:
- Scikit learn,
- Pandas,
- Numpy,
- and XGBoost.

Gemini API was incorporated into the software to incorporate the feature of Large Language Model to produce recommendations based on the context.

The application was developed and tested using Visual Studio Code as the development environment. Testing of API and debugging of its elements were done as part of the backend development to allow for effective communication between the frontend and backend elements.

The environment was developed in consideration of the flexibility and efficiency of the application in terms of machine learning workflow integration.

4.3 FRONTEND IMPLEMENTATION USING REACT

The frontend of the proposed framework was developed using React and TypeScript to create an interactive and user-friendly interface. The frontend acts as the communication layer between the user and the backend machine learning services.

The interface was designed to reduce complexities in the machine learning process by making the interaction of the users easy without necessarily having knowledge of programming languages. The frontend design was made with usability, workflow, and interactivity in mind.

Some of the frontend modules included the one responsible for the uploading and viewing of datasets. The user would upload his CSV data through the interface, and then the interface displays a preview of the datasets, complete with column descriptions and other metadata.

Column target selection is yet another aspect of functionality that can be used through the frontend. To be specific, users would be able to choose the column to be predicted based on the columns presented within the data set. Moreover, there is also the ability to exclude unnecessary columns from model training.

The frontend is responsible for initiating the metadata analysis and recommendation generation process by working with the backend APIs. After recommendation is done, the frontend will present:

- recommended machine learning models,
- contextual explanations,
- evaluation summaries,
- and training status updates.

Moreover, dashboards together with analytics and metrics pertaining to them have also been incorporated into the frontend interface. This way, users can retrieve the stats regarding model comparisons and training.

The frontend also offers the ability to download the trained models and training results. The downloaded items can be accessed through the frontend interface once the training process has been completed.

React component usage enabled the design of frontend components such as:

- upload components,
- dashboard components,
- recommendation panels,
- analytics cards, and
- result tables.

In general, the development of the frontend concentrated on ensuring simplicity, interactivity, and efficiency for the user while also ensuring that the whole process of machine learning is supported.

4.4 BACKEND IMPLEMENTATION USING FASTAPI

The backend portion of the suggested framework has been developed with FastAPI.

The role of the backend is to be the processing center that is capable of analyzing datasets, extracting metadata, performing recommendations, performing machine learning algorithms, and communicating with databases.

FastAPI was chosen due to its lightweight design, high speed, ability to handle asynchronous requests, and strong API capabilities, which are necessary when working with machine learning pipelines and Large Language Models.

The backend will have several API routes, each designed to perform a particular function within the framework. They include:

- API routes for dataset uploading,
- API routes for metadata analysis,
- Recommendation API routes,
- API routes for model training,
- Dashboard summary API routes, and
- Artifact download API routes.

After uploading the data to the backend through the frontend interface, the data set is analyzed by utilizing programming languages such as Python, Pandas, and Numpy. Some of the metadata that can be extracted from the dataset include:

- Feature metadata,
- Missing values,
- Statistical metadata,
- Dimensions, and
- Target distribution.

The backend also handles target selection and feature exclusion workflows. Updated dataset configurations are processed dynamically and re-analysis can be performed without requiring dataset re-upload.

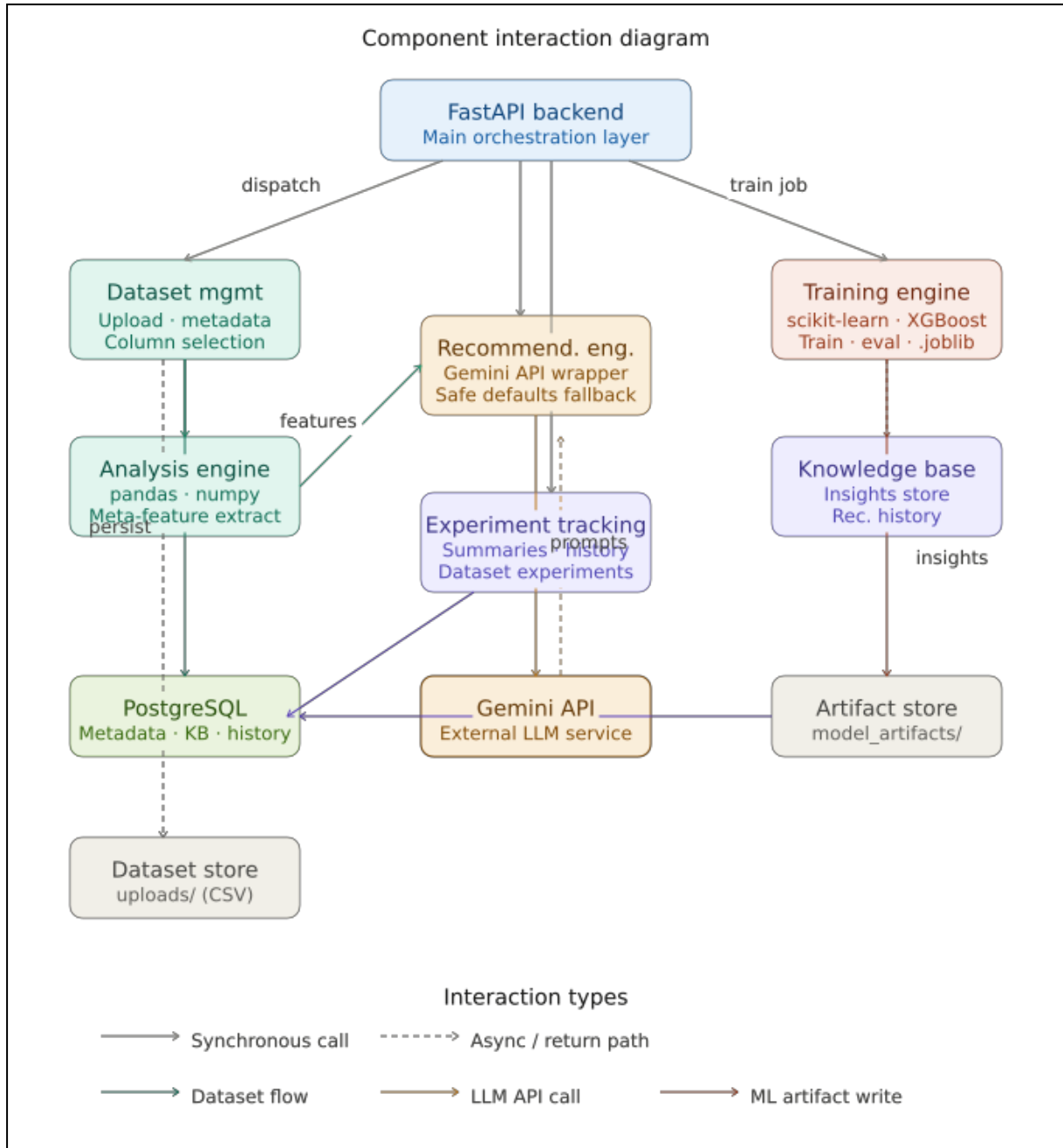


Fig 4. Component Interaction

Other capabilities of the backend include the automatic training pipeline for machine learning models. Pre-trained machine learning models recommended by the system are generated using Scikit learn and XGBoost libraries. These tasks are performed automatically by the backend component.

The evaluation of the models based on metrics such as accuracy, precision, recall, F1-score, and ROC-AUC can be performed automatically during training.

Additionally, the backend controls knowledge base persistence through PostgreSQL. Recommendations' historical performance and learning outcomes will be stored to enable adaptive learning and enhanced recommendations in the future.

In conclusion, the backend design acts as the core engine for the recommended system and allows for seamless integration of the frontend, machine learning pipeline, and recommendation service from the LLM.

4.5 POSTGRESQL DATABASE INTEGRATION

PostgreSQL's role in the proposed architecture is intended to be used for structural storage of data, including datasets, recommendation history, evaluation outcomes, and training outcomes.

Database level plays an important role for ensuring persistence across various phases of the process. Unlike the temporary nature of recommendation processing, the proposed architecture can ensure persistence of historical data for further analysis and learning.

Data is saved in the database in the form of:

- metadata for uploaded datasets
- recommendations created
- configuration parameters
- performance measures
- summary reports
- and knowledge base data

```

cursor.execute("""
CREATE TABLE IF NOT EXISTS compassllm_experiment_runs (
    run_id UUID PRIMARY KEY,
    dataset_id UUID,
    mode TEXT NOT NULL,
    task_type TEXT,
    target_column TEXT,
    model_name TEXT NOT NULL,
    model_category TEXT NOT NULL,
    parameters JSONB,
    selected_features JSONB,
    preprocessing JSONB,
    metrics JSONB,
    instruction TEXT,
    status TEXT NOT NULL,
    error_details TEXT,
    train_time DOUBLE PRECISION,
    inference_time DOUBLE PRECISION,
    run_metadata JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
""");
conn.commit()

cursor.execute("""
CREATE TABLE IF NOT EXISTS compassllm_generated_knowledge (
    knowledge_id SERIAL PRIMARY KEY,
    run_id UUID,
    dataset_id UUID,
    instruction TEXT NOT NULL,
    metadata JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
""");
conn.commit()

```

Fig5. Script for data profiling (a)

```

        created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
    );
    """);
    conn.commit()

    cursor.execute("""
CREATE TABLE IF NOT EXISTS compassllm_generated_knowledge (
    knowledge_id SERIAL PRIMARY KEY,
    run_id UUID,
    dataset_id UUID,
    instruction TEXT NOT NULL,
    metadata JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
""");
    conn.commit()

    logger.info('COMPASSLLM schema initialized successfully')
except Exception:
    if conn is not None:
        conn.rollback()
        logger.exception('Failed to ensure COMPASSLLM schema')
        raise
finally:
    if conn is not None:
        release_connection(conn)

initialize_connection_pool()
ensure_compassllm_schema()

```

Fig5. Script for data profiling (b)

```

# =====
# SAMPLE EXECUTION: Run the complete metadata collection pipeline
# =====

# Step 1: Configure for your dataset
DATASET_PATH = "BreastCancer.csv" # Change to your dataset path
MODE = "supervised" # 'supervised' or 'unsupervised'
TARGET_COLUMN = "diagnosis" # Change to your target column for supervised mode

# Step 2: Run the pipeline
logger.info('=' * 80)
logger.info('Starting COMPASSLLM Metadata Collection Pipeline')
logger.info(f'Dataset: {DATASET_PATH}')
logger.info(f'Mode: {MODE}')
logger.info(f'Target Column: {TARGET_COLUMN}')
logger.info('=' * 80)

try:
    summary = run_pipeline(DATASET_PATH, MODE, TARGET_COLUMN)
    logger.info(f'Pipeline completed successfully with {len(summary)} experiments')
except FileNotFoundError as e:
    logger.error(f'Dataset not found: {e}')
except Exception as e:
    logger.exception(f'Pipeline execution failed: {e}')

# Step 3: Display results
if 'summary' in locals() and not summary.empty:
    print('\n' + '=' * 80)
    print('EXPERIMENT SUMMARY')
    print('=' * 80)
    print(summary[['model_name', 'task_type', 'status']].value_counts())
    print('\n' + '=' * 80)
    print('BEST MODELS BY TASK TYPE')
    print('=' * 80)

```

Fig5. Script for data profiling (c)

```

from psycopg2.pool import ThreadedConnectionPool
from psycopg2 import sql

POOL = None

def initialize_connection_pool(minconn: int = 1, maxconn: int = 10):
    global POOL
    if POOL is None:
        POOL = ThreadedConnectionPool(
            minconn,
            maxconn,
            dsn=PG_DATABASE_URL,
        )
    return POOL

def get_connection():
    if POOL is None:
        initialize_connection_pool()
    return POOL.getconn()

def release_connection(conn):
    if POOL is not None and conn is not None:
        POOL.putconn(conn)

def ensure_compassllm_schema():
    """
    Create COMPASSLLM-specific metadata tables.
    NOTE: Backend tables (datasets, experiments, knowledge_base_entries) already exist in the database.
    This function creates additional tables for comprehensive metadata collection and instruction generation.
    """
    conn = None
    try:
        conn = get_connection()
    except:
        pass
    if conn is not None:
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS datasets (
                dataset_name VARCHAR(255) PRIMARY KEY,
                dataset_path VARCHAR(255),
                mode VARCHAR(50),
                target_column VARCHAR(255),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );
            CREATE TABLE IF NOT EXISTS experiments (
                experiment_id SERIAL PRIMARY KEY,
                dataset_name VARCHAR(255) REFERENCES datasets(dataset_name),
                mode VARCHAR(50),
                target_column VARCHAR(255),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );
            CREATE TABLE IF NOT EXISTS knowledge_base_entries (
                entry_id SERIAL PRIMARY KEY,
                dataset_name VARCHAR(255) REFERENCES datasets(dataset_name),
                mode VARCHAR(50),
                target_column VARCHAR(255),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            );
        """)
        conn.commit()
        cursor.close()
    finally:
        if conn:
            release_connection(conn)

```

Fig5. Script for data profiling (d)

Adaptive learning is one of the major advantages provided by the use of PostgreSQL. Recommendations that have been made previously can be analyzed to improve future recommendations.

Workflow management is yet another benefit that comes from the application of database integration. This is because previous work is saved, hence there will be no need to train the model again.

Some other benefits of PostgreSQL are the structure of queries and data search efficiency for dashboard and recommendation tracking. It is possible for the back end to communicate with the PostgreSQL database using database integration and ORM communication. In general, PostgreSQL allows effective data management through database integration.

4.6 RECOMMENDATION AND TRAINING WORKFLOW

The pipeline for recommendation and training in the proposed system was designed in such a way that it would help to automate the process of selecting a model using the machine learning algorithm with the least amount of experimentation through context-based reasoning. First of all, a user has to upload a dataset using the frontend application. After uploading the dataset, the backend takes care of extracting metadata from the dataset. This process involves gathering important information regarding the dimensions of the dataset, features, null values, statistics, targets, among others.

Once this is done, the next step is where the users get to select their target column based on their predictive intentions. Additionally, there is the option of dropping features which can be helpful in reducing the amount of information being trained on.

The meta data along with the record of the dataset as well as the user's commands is then forwarded to the recommendation engine that will employ the use of Gemini in determining the most suitable ML algorithm. In contrast with the conventional approach of running experiments in a large space, the proposed framework will offer a smart approach of restricting the recommendation space before training commences.

After recommendation is complete, the backend will trigger the automatic process of training the machine learning algorithms. Machine learning packages such as Scikit learn and XGBoost

are used to train the recommended algorithms generated by the backend. Missing values treatment, categorical variable encoding, and data splitting will all occur automatically at this stage.

After training all models, the system will evaluate them using appropriate metrics. The system will select the model with the best performance result and display it to the user using the dashboard interface. Training results, recommendation reasons, and evaluation scores will also be displayed. The final step in the workflow is storing recommendations and training results in the knowledge base.

4.7 EXPERIMENTAL SETUP

The experiment was aimed at assessing how the proposed Context-Oriented ML Model Prediction and Adaptive Selection System would perform in various machine learning experiments and datasets.

In implementing the solution, the following tools were used: full-stack development technology based on web and Python-based machine learning libraries; React + TypeScript for building the frontend component; FastAPI for developing the back-end services; and PostgreSQL for storing metadata, recommendations, and training results.

The machine learning pipeline is realized based on:

- Scikit learn library,
- Pandas library,
- NumPy library,
- and XGBoost library.

The use of Gemini API for the integration of the Large Language Model provided the context recommendation functionality for the developed system.

The analysis was carried out by performing experiments using tabular datasets of various fields. The diversity of dataset sizes, features, missing data, and distribution of the target variable was chosen to study the adaptability of the proposed approach.

The workflow in the system comprised:

- uploading the data set,
- extracting metadata,
- selecting target variable,
- excluding features,
- generating recommendation,
- training automatically,
- evaluation, and
- visualization using dashboard.

Machine learning recommendations were automatically generated for all data sets based on contextual reasoning of the Gemini model. Comparisons between the framework and conventional AutoML systems were also considered in terms of computational efficiency and recommendation efficiency.

The main purpose of conducting experiments was to test if contextual reasoning by large language models could minimize unnecessary trial-and-error processes without compromising the quality of model recommendations.

4.8 EVALUATION METRICS

The use of evaluation metrics is very crucial when considering assessing the performance of the model in machine learning and recommendation systems. Various evaluation metrics have been used in this framework to assess the effectiveness and prediction accuracy of the model.

In case of classifications, the following evaluation metrics were applied:

Accuracy

Accuracy refers to the ratio between the number of accurate predictions made and the total number of predictions made.

Precision

Precision is defined by the ratio between the number of accurate positive predictions made and the total number of positive predictions made.

Recall

Recall represents the percentage of true positives that have been accurately predicted with respect to the total number of actual positives. Recall is an appropriate metric to use when detecting all positive instances is significant.

F1-Score

This is a combination of both the precision and recall scores and thus is a measure of model performance.

ROC-AUC

Apart from using prediction metrics, other metrics were also used to evaluate the efficiency of the recommender system framework.

Computational Time

Time taken for the execution of recommendation generation and learning models was calculated to assess the computational efficiency.

Recommendation Relevance

Relevance of the recommended algorithms was analyzed using comparison between the recommendation outputs and the final learning model performance.

Resource Usage

Usage of CPU and efficiency of overall process flow was observed to assess the reduction of computational overhead.

The following evaluation metrics enabled assessment of both machine learning performance and practical efficiency of the recommendation approach.

4.9 EXPERIMENTAL RESULTS AND ANALYSIS

The outcome of the experiment showed that the designed architecture could develop context-aware machine learning recommendations and automated machine learning models training.

The designed architecture was able to process uploaded data, extract metadata, generate recommendations through the integration of Gemini API, and automate machine learning models training. It was found that several datasets processed by the designed architecture identified the appropriate machine learning algorithms with minimal effort.

The dashboard component illustrated the metrics used for the evaluation process, provided summaries on the training process, and explained recommendations to the user, thus increasing the workflow efficiency. The results of the experimental study demonstrated that context-aware recommendations contributed to minimizing excessive testing compared to standard brute force AutoML methods since it did not consider many irrelevant models for evaluation.

The effectiveness of the adaptive learning approach through the knowledge base was another key finding in this experiment. The historical recommendation database and training results contributed to consistent recommendation processes in the future. On the whole, the experimental findings confirmed the real-world feasibility of combining Large Language Models with ML automation systems.

4.9.1 Model Recommendation Accuracy

The suggested recommendation model showed high accuracy in recommending appropriate machine learning models for various data sets.

In the majority of experiments, the algorithms recommended by the Gemini-based recommendation engine performed similarly to their competitors once they were trained. The recommendation engine was able to recommend appropriate models taking into consideration the nature of data sets, including the distribution of features, classification, missing data points, and complexity of the data set.

Decision tree-based algorithms like Random Forest and XGBoost were among the commonly recommended algorithms for tabular data sets with mixed features and class imbalance. On the other hand, Logistic Regression and Decision Tree models were recommended for smaller and simpler data sets.

4.9.2 Comparison With Traditional Automl

Comparison of the proposed framework with traditional AutoML processes was also made to identify differences between efficiency and behavior when recommending. Traditionally, AutoML approaches utilize an approach that involves the exhaustive experiments with different algorithms and combinations of hyperparameters. Although this type of process leads to achieving efficient models, such models usually require substantial computation time.

Unlike traditional AutoML processes, the suggested framework tried to reduce the number of experiments prior to model creation based on Gemini recommendations. This way, unnecessary models were not created.

Experimental results indicated that the proposed approach helped to cut down unnecessary experimentation without compromising the effectiveness of predictions. It facilitated an efficient process and made the training simpler. Another crucial feature of the proposed method was its explainability. Standard AutoML frameworks operate on the principles of black boxes, but the proposed model provided human-understandable interpretations for recommendation decisions.

4.9.3 Computational Cost Reduction

Reducing the computational load was one of the primary goals of the study.

The experimental analysis revealed that the use of context-driven recommendations led to considerable savings on redundant model evaluations. AS a result of the approach that emphasized the importance of relevant algorithms prior to their training, computation time and load were considerably lower than for the brute force technique.

The decrease in experimentation helped in optimizing resource usage as well. The amount of CPU processing and workflow intricacy was less when compared with the brute-force experimentation processes.

The optimization provided in the above process will be advantageous for people who work in environments where there are limited computing resources available such as universities and startups.

4.9.4 Adaptive Recommendation Performance

The adaptive recommendation algorithm via the knowledge base had a positive effect on achieving consistency in workflows and refining recommendations.

The system collected data on recommendation results, training metrics, and experimental outcomes after each run. This data could be used to refine recommendations in the future by understanding the relationship between dataset attributes and model performance.

With more datasets, the system became better at recommending appropriate algorithms. The adaptive nature of the system made it evolve from a simple recommendation system into an intelligent recommendation framework.

Adaptive learning was able to improve scalability in the enhancement of the system in the future. Various other ways in which the recommendation optimizations can be fine-tuned can be added in the system using the historical data collected.

In general, the performance of adaptive recommendation proved the ability to combine LLMs with learning mechanisms in order to create intelligent recommendation machine learning systems.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

As the number of machine learning applications using machine learning algorithms grows, it's becoming more important than ever to make machine learning pipelines efficient and intelligent. While there have been efforts to automate some of the process of creating machine learning models using AutoML tools, a lot of traditional machine learning methods have lots of experimentation and computation. Moreover, none of the current recommendation engines currently have the contextual, explainable and adaptive properties.

The problems mentioned above are to be solved by the thesis on which a Contextual system based on Large Language Models for the prediction and Adaptive selection of ML models is suggested. The main focus of the research was to design and integrate components like contextual reasoning, automaion of the logistical aspects of the machine learning process, adaptivity of the recommendation and explanation aspect.

The proposed architecture focused on the following functions related to the dataset: Upload, extract metadata, provide machine learning recommendations based on Large Language Models, automatic training of models, analysis of prediction accuracy, and saving previous results to be used later for learning. This is not the usual "run a big number of models with no discrimination" approach of virtually every AutoML solution, but was a bid to minimize the need for a lot of guessing.

The development of the mentioned architecture was a prototype to prove that this kind of integration is possible. This platform had the capability of:

- Dataset uploading & previewing,
- Metadata extraction,
- Column selection,
- Feature removal,
- Generating recommendations with the LLMs, and
- Model training,

- Evaluation & dashboarding,
- Downloadable model artifacts,
- Learning database management.

The experimental finding show that we can save the computational cost and be efficient and cost-effective in terms of computations to be generated by the context to make recommendations compared to the approaches in AutoML that involve finding them by brute force. Advantage of this, when coupled with adding the Gemini API to the architecture, were the ability to build recommendations based on context, using properties of the dataset.

An additional key success to the new approach is the explainability. As the computer fluency explanation was in natural language, it assisted users to gauge which algorithms had been recommended and why.

Both aspects introduced more complexity as they have ensured that recommendations given by the recommendation system will be based on the previously known experience and it will even become more consistent. That is possible thanks to the adaptability of the recommendation system.

5.2 SUMMARY OF CONTRIBUTIONS

The suggested study played an essential role in uplifting the field of intelligent machine learning automation and contextual awareness based recommendation system.

First, the following contributions can be attributed to this research work: Design of the Context-Oriented ML Model Prediction & Adaptive Selection System, Large Language Models and Automated machine learning pipelines. It is a single system that combines all the following features: contextual reasoning, meta data analytics, automated modeling, adaptive learning and explanation capabilities.

Apart from this, an important attribute of this research work is the use of large language models to generate a context-based recommendation. This is because available AutoML frameworks rely on brute force search techniques, and the recommended framework leverages metadata, sample datalists and user input to create recommendations for machine learning.

This research will enable me to make some of the computational algorithms used in machine learning more efficient. This will lead to fewer recommendation to train as filtering will be performed before training starts.

One other significant contribution is explained recommendation mechanisms similar to those included. The system can give explanations in human readable text, addressing the question of when certain algorithms might be applicable to a given data set. This will enhance transparency and assist users' comprehension of the recommendation process.

In addition, it has done a contribution to the development of an adaptive learning mechanism with a Knowledge Base of the dissertation. The results of past recommendations, as well as evaluation and training are recorded and used to improve upcoming recommendations. This enables the framework to constantly evolve over the years.

The practical implementation of the system, from front- to backend is the other contribution. Combining the technologies of React, FastAPI, PostgreSQL, Gemini API, and machine learning libraries created a modular and scalable intelligent platform, which could allow for the automation of various stages of the machine learning process.

This study also brings some contribution on the following basis; - Integration of components of workflow such as:

- dataset upload,
- metadata extraction,
- target selection,
- feature exclusion,
- recommendation generation,
- automated training,
- dashboard analytics,
- and artifact generation

in the kind of a unified environment. This makes them easier to use, and makes the machine learning development process easier.

Finally, during the research the effectiveness of contextual recommendation systems was also experimentally evaluated, which showed that it could be effective for boosting the efficiency of the workflow and lower the computational cost over traditional AutoML approaches.

Overall, the dissertation works towards the progress of intelligent, adaptive, explainable and context-aware machine learning recommendation systems, that would help modern software engineering in an AI-assisted setting.

The advantages of proposed system are described below:

The proposed framework has several benefits over the conventional machine learning and AutoML systems. One of the major features of the proposed system is the contextual recommendation. As LLMs are incorporated into the recommendation workflow, the framework will be able to consider the various features of the datasets and then make smarter recommendations – rather than just trying the recommendations.

5.3 LIMITATIONS

The suggested system showed strong effectiveness and usefulness, but there are some drawbacks in the present system. The framework faces some drawbacks, like relying on outputs from Large Language Models. Recommendations come from reasoning within a context using the Gemini API, thus the recommendations' quality could change for certain situations because of poor billing prompts and model interpretation. The recommendations given for certain cases might not be the best ML solution for the global one.

The first is, the current implementation mostly focuses on structured tabular data. The advanced support of unstructured information (images, audio, video and deep learning workflows of textual information) is not considered in the current research.

The integration of this system can also take place through the use of API calls to the internet to get recommendations from LLM. It follows that in such scenarios, the availability of APIs and timely responses are essential for these recommendation processes.

However, there is one limitation regarding the lack of advanced hyperparameter optimization methods. This is because currently, the system has intelligent model recommendations and automatic model training but lacks any kind of hyperparameter tuning process.

The adaptive learning capability implemented in the system is still at its early stages where it simply keeps records of the recommendations in the past while excluding simplistic knowledge bases. The more advanced capabilities in terms of self-learning and reinforcement-based optimizations were not implemented in this study. Another possible source of scaling problem could come from very large data sets and work processes utilized simultaneously by several users yet not supported by any distributed system infrastructure.

This is the other constraint that we have, known as explainability consistency. Even though the system can generate human-understandable explanations for the recommendations provided, these explanations will differ in quality and complexity depending on the amount of context information available to the Large Language Models.

Last but not least, it is important to note that the suggested framework was tested mainly on the benchmark dataset under an experimentally organized setting. In the current study, there was no attempt made to scale the technological application and test the product in its real-life environment.

Notwithstanding these limitations, the suggested framework revealed the potential of applying LLMs alongside machine learning tools in the context of intelligent and adaptive recommendation work.

5.4 FUTURE SCOPE

The proposed Context-Oriented ML Model Prediction and Adaptive Selection System shows the potential of using Large Language Models with machine learning automation workflows in real-world scenarios. The proposed system is able to generate recommendations based on context, train, evaluate, and adapt automatically, but it has some limitations and can be further developed in the future to address the following issues: Contextual recommendations for generating recommendation is successfully done, but its generation is not yet fully automated; Training, evaluation and adaptive learning is successfully done automatically, but there are still some limitations; The framework can be expanded by developing more advanced evaluation methods for training and adaptive learning.

5.4.1 Gpt-Based Recommendation Enhancement

Use of Gemini API integration to generate the recommendations is employed in this particular case, although it is possible to employ advanced models and multi-agent reasoning systems based on GPTs in order to further improve the quality of recommendations in future versions of the system.

Indeed, recently, the GPT models showed very good results in various tasks such as reasoning, coding, and analysis tasks due to their high-contextual capability. GPT models can also be used to create very precise recommendations in machine learning by taking into account the area of interest.

In addition to the above, recommendation systems in the future will facilitate interactions with the system on a conversational basis, where users will be able to directly ask queries to the system. Users will be capable of stating a particular issue with the data set, getting intelligent recommendations about pre-processing and training of workflows.

Another possible improvement to the system is that more than one LLM agent can function as a group. Some of the agents specialize in:

- dataset understanding,
- preprocessing recommendation,
- model selection,
- evaluation analysis,
- and optimization strategies.

This multi-agent recommendation method could help boost the reliability of the model and reduce the outputs inconsistency from the single model. GPT systems can further enhance explainability by providing more detailed explanations and suggestions for the steps involved in the reasoning. This would be more convenient for both newbies and advanced programmers to develop machine learning.

5.4.2 Continuous Learning And Fine-Tuning.

A basic knowledge management system has also been developed that captures the results of the recommendations and previous trainings. But in the future, these systems can greatly enhance

their adaptive learning capabilities using ongoing learning and tuning systems. Another way to make the existing system better is to tune the recommendation models based on the specific domain. These models can then be tuned to become recommendation systems for domains such as healthcare, finance, cyber security, education, and industrial analytics data sets.

In addition to the above-mentioned techniques, the other ways that adaptive learning could be employed include reinforcing learning algorithms that will help improve the system's capabilities in terms of constantly improving recommendations through reinforcement learning. In summary, the framework can be made smarter and capable of self-enhancement through continuous learning and improvement.

5.4.3 Joint project with CRFS for the Multi-Domain Benchmark Dataset Expansion

The main test cases for the current implementation involved the use of structured benchmark datasets in selected domains. In future research, there could be a need to develop more ways of evaluating the recommendation engine by applying it to more and larger datasets.

It would be useful for evaluating the recommendation engine by using other datasets that involve health care, financial systems, education, cybersecurity, environmental studies, social media analysis, and IoT systems.

Additional "expansions" of the data sets might include:

- large real-world data sets,
- noisy data sets,
- high-dimensional data sets,
- data streams, and
- imbalanced data sets.

The ensuing evaluations would assist in achieving further understanding regarding the strengths and weaknesses of contextual recommender systems when faced with different types of real-world applications.

Another area for further research would be to create benchmark repositories which are specifically designed for benchmarking LLM-based AutoML systems. It will be helpful if the data sets used for each standard benchmark can be provided to researchers for benchmarking

other recommender systems in a standardized manner. Furthermore, a large-scale data set would help improve the learning capacity of the knowledge base as well as improve prediction/recommendation accuracy.

5.4.4 Explainable AI-Based Recommendations

The need for explanation is very essential especially when it comes to making decisions in places whereby the decision taken by machine learning can bring great changes to individual or organizational lives. While the current explanation system gives explanations for some basic reasoning, future systems that will come after this one could be better in explaining their decisions. Such explanation systems could give:

- Recommendation reasons,
- Importance ranking of features,
- Confidence in results,
- and workflow clarity.

Also, the relationships between certain properties of the data sets and the recommended algorithms could be presented visually using an interactive dashboard in the future.

Providing explanations that are user-friendly for end-users who lack technical expertise is another key future trend. Popular AI software applications provide results that may not be comprehensible to laymen. User-friendly explanations can include natural language descriptions, and support for the guided process is feasible in future systems. For example, in critical domains such as healthcare and banking, where transparency and understanding are crucial, Explainable AI can improve the reliability and acceptance of AI-generated suggestions.

5.5 OVERALL CLOSING REMARKS

The study examined the adoption of LLM+ML to develop a Context Oriented ML Model Prediction and Adaptive Selection System, which could generate intelligent recommendations for machine learning models.

It was found that utilizing contextual reasoning in addition to automated machine learning can help minimize unnecessary testing, offer an explanation for the generated recommendations, and improve the process of developing machine learning solutions. The system design

incorporated the analytical processing of data, metadata generation, recommendation creation, automated training, dashboard analytics, and adaptive learning in one integrated system.

One of the crucial results of the study is that Large Language Models will not only be able to assist in conversational applications but also make a contribution in other areas. Large Language Models' intelligence can be utilized in machine learning and software engineering decision-making processes.

This study proposes a new framework that could help create an adaptive, explainable, and effective system of Artificial Intelligence assistance. There are some limitations in the current version of the model. However, the study indicates that the use of Large Language Models (LLMs) in context-aware recommendation systems could bring many future opportunities.

With the growing complexity of machine learning implementations, intelligent recommendation systems are set to gain greater importance for process optimization, reduction of computing resources, and greater accessibility of AI technologies to a broader population. In general, the present dissertation represents an effort towards building smarter machine learning automation systems with human-centered design and can be viewed as offering several directions for future research.

APPENDIX

A- DATASET DETAILS

We have created a benchmarking dataset to validate the results that the system are producing and to find tune the LLM recommendations. The Dataset that has been created is as follows:

S. No	Datas et Category	Dataset	Data Type	Featu res	Sam ples	Problem Type	Best Workin g Models	Accura cy (Avera gein %)	Precis ion (Aver age in %)
1	Small Tabul ar	Iris	Numeri cal	4	150	Classific ation	Neural Network classifica tion	97.368	97.43 6
2	Small Tabul ar	Wine	Numeri cal	13	178	Classific ation	Random forest	100%	100%
3	Small Tabul ar	Breast Cancer Wiscons in	Numeri cal	30	569	Classific ation	Random forest	97.90	97.94
4	Small Tabul ar	Glass Identific ation	Numeri cal	9	214	Classific ation	XGboost	85.185	93.82 5
5	Small Tabul ar	Seeds	Numeri cal	7	210	Classific ation	xgboost	87.2%	83.83

6	Mixed Tabular	Adult Income	Mixed	14	4884 2	Classification	XGBOOST	87.225	83.38 3
7	Mixed Tabular	Bank Marketing	Mixed	16	4521 1	Classification			
8	Mixed Tabular	Mushroom	Categorical	22	8124	Classification	Xgboost, svm, random forest	100%	100%
9	Mixed Tabular	Heart Disease	Mixed	13	303	Classification	Xgboost, logistic regression	81.575	83.18 5
10	Mixed Tabular	German Credit	Mixed	20	1000	Classification	Random forest	78	76.17 9

B- GITHUB LINK:

<https://github.com/aarsee21/COMPASSLLM>

C- REFERENCES

- [1] A. Tornede, L. Gehring, T. Tornede, M. Wever and E. Hüllermeier, “Algorithm Selection on a Meta Level,” *arXiv preprint arXiv:2107.09414*, 2021.
- [2] G. Nguyen, M. J. Islam, R. Pan and H. Rajan, “Manas: Mining Software Repositories to Assist AutoML,” in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, Pittsburgh, PA, USA, 2022.
- [3] G. Nápoles, I. Grau, Ç. Güven, O. Özdemir and Y. Salgueiro, “Which is the Best Model for My Data?,” *arXiv preprint arXiv:2210.14687*, 2022.
- [4] Y. Shen, Y. Lu, Y. Li, Y. Tu, W. Zhang and B. Cui, “DivBO: Diversity-aware CASH for Ensemble Learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] N. Hollmann, S. Müller and F. Hutter, “Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering,” in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [6] S. Zhang, C. Gong, L. Wu, X. Liu and M. Zhou, “AutoML-GPT: Automatic Machine Learning with GPT,” *arXiv preprint*, 2023.
- [7] A. Tornede, D. Deng, T. Eimer, J. Giovanelli, A. Mohan, T. Ruhkopf, S. Segel, D. Theodorakopoulos, T. Tornede, H. Wachsmuth and M. Lindauer, “AutoML in the Age of Large Language Models: Current Challenges, Future Opportunities and Risks,” *arXiv preprint arXiv:2306.08107*, 2024.
- [8] L. Zhang, Y. Zhang, K. Ren, D. Li and Y. Yang, “MLCopilot: Unleashing the Power of Large Language Models in Solving Machine Learning Tasks,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024.

- [9] Y. She, S. Biswas, C. Kästner and E. Kang, “FairSense: Long-Term Fairness Analysis of ML-Enabled Systems,” 2024.
- [10] J. Simões and J. Correia, “EDCA – An Evolutionary Data-Centric AutoML Framework for Efficient Pipelines,” *arXiv preprint arXiv:2503.04350*, 2025.
- [11] X. Zhang, J. Zhai, S. Ma, X. Guan and C. Shen, “DREAM: Debugging and Repairing AutoML Pipelines,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 4, 2025.
- [12] H. Ye, J. Li, H. Zhao, D. Guo and Y. Chang, “LLM Meeting Decision Trees on Tabular Data,” in *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [13] C. Spiess, M. Vaziri, L. Mandel and M. Hirzel, “AutoPDL: Automatic Prompt Optimization for LLM Agents,” *AutoML Conference*, 2025.
- [14] M. Garouani, “An Experimental Survey and Perspective View on Meta-Learning for Automated Algorithms Selection and Parametrization,” *arXiv preprint*, 2025.
- [15] B. Xu, K. Ding, W. Liu, Y. Lu and B. Cui, “CoFEH: LLM-driven Feature Engineering Empowered by Collaborative Bayesian Hyperparameter Optimization,” *ACM Conference Proceedings*, 2026.
- [16] <https://archive.ics.uci.edu/>

2403030020_CONTEXT-ORIENTED ML MODEL PREDICTION AND ADAPTIVE SELECTION SYSTEM USING LLMS

ORIGINALITY REPORT

3%

SIMILARITY INDEX

2%

INTERNET SOURCES

1%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1

arxiv.org

Internet Source

<1%

2

Yinglin Xia, Jun Sun. "Machine Learning for
Microbiome Statistics", CRC Press, 2026

Publication

<1%

3

ebin.pub

Internet Source

<1%

4

jlist.ir

Internet Source

<1%

5

open-innovation-projects.org

Internet Source

<1%

6

"Smart Business Technologies", Springer
Science and Business Media LLC, 2026

Publication

<1%

7

"Applications of Evolutionary Computation",
Springer Science and Business Media LLC,
2026

Publication

<1%

8

www.ijcstjournal.org

Internet Source

<1%

9

ijrpr.com

Internet Source

<1%

10

Submitted to Dublin Business School

Student Paper

<1%

11	www.mdpi.com Internet Source	<1 %
12	"Applications of Evolutionary Computation", Springer Science and Business Media LLC, 2025 Publication	<1 %
13	"Rough Sets", Springer Science and Business Media LLC, 2024 Publication	<1 %
14	scienceportal.tecnalia.com Internet Source	<1 %
15	theses.hal.science Internet Source	<1 %
16	www.researchsquare.com Internet Source	<1 %
17	Nguyen, Giang. "Fairness Specification and Repair for Machine Learning Pipeline.", Iowa State University Publication	<1 %
18	Xun Huang, Rongwen Yao, Yunhui Zhang, Xiao Li, Zhongyou Yu, Hongyang Guo. "A novel irrigation water quality index (PCIWQI) assessment and prediction: insights into model performance and computational efficiency from TabPFN and Tree-Based models", Journal of Environmental Chemical Engineering, 2025 Publication	<1 %
19	baadalsg.inflibnet.ac.in Internet Source	<1 %
20	www.researchgate.net Internet Source	<1 %